

Infrastrutture - Reference Architecture

Architettura di riferimento dell'Infrastruttura di
Azienda Zero

Data

9 Oct 2025

Versione

1.1.0

Copyright

Questo documento appartiene ad Azienda Zero. I contenuti del medesimo – testi, tabelle, immagini, etc. – sono protetti ai sensi della normativa in tema di opere dell'ingegno. Tutti i diritti sono riservati. Il presente documento potrà essere utilizzato per la realizzazione di progetti regionali liberamente ed esclusivamente nel rispetto delle regole (standard) stabilite da Azienda Zero. Ogni altro utilizzo, compresa la copia, distribuzione, riproduzione, traduzione in altra lingua, potrà avvenire unicamente previo consenso scritto da parte di Azienda Zero. In nessun caso, comunque, il documento potrà essere utilizzato per fini di lucro o per trarne una qualche utilità.

ATTENZIONE

Per verificare la presenza di una versione aggiornata di questo documento contattare sistemi.informativi@azero.veneto.it.

Registro delle Revisioni

Data	Versione	Descrizione	Nominativo
05/2022	1.0.1	Prima Stesura	Azienda Zero
09/2022	1.0.2	Revisione	Azienda Zero
10/2022	1.0.3	Revisione	Azienda Zero
11/2022	1.0.4	Revisione	Azienda Zero
12/2022	1.0.5	Revisione	Azienda Zero
01/2023	1.0.6	Aggiunti paragrafi: • Docker Base Image • Mobile Application	Azienda Zero
03/2023	1.0.7	Resilienza e Affidabilità	Azienda Zero
04/2023	1.0.8	Affidabilità, Resilienza e Continuità Operativa Ambienti di Test Protocolli di comunicazione	Azienda Zero
09/2023	1.0.9	Revisione generale di forma	Azienda Zero
01/2024	1.0.10	Inserimento capitolo su invio mail	Azienda Zero
02/2024	1.0.11	Soluzioni CMS	Azienda Zero
05/2024	1.0.12	Protocolli di comunicazione Certification Authority e Certificati mTLS e TLS	Azienda Zero
05/2024	1.0.13	Exit strategy da soluzioni SaaS	Azienda Zero
08/2024	1.0.14	Limiti di accesso a base dati di altri servizi	Azienda Zero
10/2024	1.0.15	Linee Guida ACN sulla Crittografia	Azienda Zero
12/2024	1.0.16	Aggiornato capitolo su Audit	Azienda Zero
07/2025	1.1.0	Revisione e pubblicazione	Azienda Zero

Sommario

1 INTRODUZIONE	9
1.1 Infrastruttura Tecnologica di Azienda Zero	9
1.1.1 Resilienza e Affidabilità	10
1.2 Presa in carico di nuove applicazioni	10
1.2.1 Assegnazione di uno o più Account AWS	10
1.2.2 Presa in carico di Account AWS	11
1.2.3 Consegna di software precompilato	11
	11
2 CONCETTI DI BASE SULL'ARCHITETTURA A MICROSERVIZI	12
2.1 Individuazione dei microservizi	12
2.2 Architettura logica e architettura fisica	13
2.3 Sovranità dei dati per microservizio	13
2.3.1 Lettura di dati di competenza di più microservizi	14
2.3.2 Scrittura dati su più microservizi	15
2.4 Asincronia nell'architettura a microservizi	15
2.5 Comunicazione tra microservizi	16
2.6 ROA e SOA	17
2.6.1 Caching in REST	18
	18
3 ARCHITETTURA APPLICATIVA DI RIFERIMENTO	19
3.1 Front end	21
3.2 Back end	21
3.2.1 API Gateway e API Management	21
3.2.2 Microservizi	22
3.2.3 Backend For Frontend (BFF)	22
3.3 Mobile Application	22
3.4 Soluzioni CMS	23
3.5 Authentication Manager	23
3.6 Gestione Eventi/Messaggi	24
3.7 Log Management	24
3.7.1 Audit	24
3.7.2 Analytics	25
3.7.3 Debug	25
3.8 Messaggistica WebSocket	26

3.9 Coreografia/Orchestrazione	27
3.9.1 Transactional Outbox Pattern	28
3.10 Schedulazione	28
3.11 Comunicazioni Telefoniche	29
	29
4 SISTEMI DI AUTENTICAZIONE	30
4.1 Integrazione col sistema di autenticazione APMS	31
4.1.1 Componente di profilazione di APMS	31
4.1.2 Registrazione WAYF	32
4.1.3 Token Autorizzativo (ID_Token)	32
4.1.4 Ulteriori approfondimenti su APMS	33
4.2 SAML Token	34
4.2.1 Accesso ai servizi infrastrutturali	34
4.3 Google Cloud Identity	34
	34
5 VINCOLI PROGETTUALI	35
5.1 Strategia Cloud Italia, Linee Guida AgID e ACN	35
5.2 Attestazione di conformità alle specifiche tecniche (ex processo di labeling)	35
5.3 Ambienti di Rilascio	36
5.4 Durata Sessione Utente	37
5.5 Role Based Access Control (RBAC)	37
5.6 Gestione stringhe di testo	38
5.6.1 Statiche	39
5.6.2 Testi di input	39
5.6.3 Testi di output	39
5.6.4 Messaggi	39
5.7 Gestione degli errori	41
5.8 Esito di un'operazione	42
5.9 API	43
5.10 Healthcheck	43
5.11 Interfaccia Utente	43
5.12 Feature Toggle e API Version	43
5.13 Risorse esterne e servizi terzi	44
5.14 Protocolli di comunicazione	45
5.15 Certification Authority e Certificati mTLS e TLS	45
5.15.1 Certification Authority	45

5.15.2 mTLS	46
5.16 Docker Base Image	46
5.17 Affidabilità, Resilienza e Continuità Operativa	46
5.18 Ambienti non di produzione	46
5.19 Limiti invio E-Mail	47
5.20 Limiti di accesso a base dati di altri servizi	47
5.21 Exit Strategy dai sistemi SaaS	48
6 CI/CD	49
6.1 Continuous Integration	49
6.2 Continuous/Automated Deployment	50
7 DOCUMENTAZIONE	51

Documenti applicabili

I seguenti documenti saranno disponibili su richiesta e valutazione di Azienda Zero.

Codice	Titolo
SVS	Compendio dei software infrastrutturali supportati e relative versioni

Documenti di riferimento

I documenti di competenza di Azienda Zero saranno resi disponibili su richiesta scrivendo a sistemi.informativi@azero.veneto.it

Codice	Titolo
APMS	Specifiche di integrazione autenticazione APMS-Team di sviluppo
GDL-O Sicurezza	Infrastruttura di sicurezza GDL-O Sicurezza
WAYF	Nota Integrativa Specifiche IAP e Sicurezza
RFC	Request for Change, documento per la presentazione di richieste da gestire tramite il change management
MI	Manuale di Installazione, foglio di calcolo che racchiude le informazioni tecniche sull'infrastruttura del servizio, la parametrizzazione e configurazione dei componenti.
SA	Scheda Architetture, documento che descrive l'infrastruttura tecnologica del servizio
Google Identity	Using OAuth 2.0 to Access Google APIs
Branching Model e Convenzioni	GIT, GITLAB, Nexus rilasci e convenzioni
Riferimenti normativi in materia di privacy, innovazione digitale della PA e cybersecurity	• GDPR e Normativa europea e internazionale • DECRETO LEGISLATIVO 30 giugno 2003, n.196 • Linee guida del Garante • Linee Guida Agid • Regolamento Cloud per la PA • Linee guida funzioni crittografiche
Log Management	Log Management - Specifiche di Integrazione

Notazioni utilizzate

Notazione	Descrizione
[dato]; [dato].[attrib].	Tra parentesi quadre si indicano i nomi dei dati che sono necessari a completare l'esposizione dei concetti. Il concetto di [dato] ricalca approssimativamente quello di variabile nell'ambito dei linguaggi di programmazione. Si possono indicare anche gli attributi con la notazione [dato]:[attributo1].
testo	Negli esempi di messaggio, il carattere Courier New è utilizzato per scrivere un testo esattamente così come deve essere utilizzato.

Definizioni ed acronimi

Definizione	Descrizione
UI	User Interface
BFF	Backend For Frontend componente che espone l'API dedicata alla UI.
SPA	Single Page Application
MS	Microservizio
DB	Database
Bounded Context	È un concetto alla base della progettazione Domain Driven Design. In questa tecnica i sistemi complessi vengono modellati come insieme di sottosistemi più elementari, i Bounded Context, e delle loro interrelazioni.
Dominio	È il dominio di business per il quale vengono individuati i dati e le funzioni da realizzare. Il dominio può riferirsi sia all'intero sistema sia alla porzione di questo gestito da un microservizio.
URI	Uniform Resource Identifier
API	Application Programming Interface
Token Autorizzativo	Token su cui si basa il meccanismo di autorizzazione OAuth2 (si veda la letteratura di riferimento).
Risorse Protette	Sono le risorse il cui accesso è consentito solo se si è in possesso di un Token Autorizzativo.
CQRS	Command Query Responsibility Segregation
ACID	Atomicity, Consistency, Isolation, Durability
OIDC	Open ID Connect
ROA	Resource Oriented Architecture
SOA	Service Oriented Architecture
GDPR	General Data Protection Regulation

JWT	JSON Web Token
IdP	Identity Provider
RBAC	Role Based Access Control
CI	Continuous Integration
CD	Continuous Delivery
Teorema CAP	l'impossibilità di ottenere simultaneamente tutte e tre le seguenti garanzie: copy-consistency, availability e partitioning
SNS	Simple Notification Services è un servizio di messaggistica gestito di AWS
SQS	Amazon Simple Queue Service è un servizio di accodamento di messaggi distribuito
POD	gruppi di uno o più container
Timestamp	usare l'annotazione secondo standard ISO 8601 es. 2022-04-15T14:08Z
CA, TLS, mTLS, SSL	CA: Certification Authority, Ente fidato che rilascia certificati di tipo TLS TLS: Transport Layer Security, Certificato server utilizzato prevalentemente per la verifica e autenticità del servizio a cui si accede mTLS: Mutual TLS, Certificato client utilizzato per autenticarsi ad un servizio protetto tramite questa metodologia SSL: Secure Socket Layer, precedente interazione di TLS oggi deprecato
FSE	Fascicolo Sanitario Elettronico Regionale

1 INTRODUZIONE

Il presente documento è pensato con l'obiettivo di offrire ai fornitori di Azienda Zero una linea guida nella progettazione e realizzazione di applicazioni ed è indirizzato alle seguenti figure professionali:

- Architetti, Analisti e Sviluppatori del Software
- System Designer
- Progettisti di Architettura di Sistemi
- Project Manager e Capi Progetto

Azienda Zero auspica che questo documento venga visto come un'opportunità per i fornitori, i quali adattandosi al suo contenuto, possono agevolare la transizione dalla fase di sviluppo a quella di messa in produzione. È altresì essenziale considerare questo documento come un punto di partenza anziché di arrivo. Eventuali miglioramenti, nuove tecnologie e metodologie proposte per potenziare il servizio saranno oggetto di attenta valutazione.

Di conseguenza, è importante percepire questo documento come dinamico e soggetto a periodici aggiornamenti. Prima di intraprendere un nuovo progetto di sviluppo, modifica evolutiva o correzione, il fornitore è tenuto a richiedere la versione più aggiornata del documento.

Per condividere commenti, segnalazioni o suggerimenti, si prega di scrivere all'indirizzo: change.it@azero.veneto.it

1.1 Infrastruttura Tecnologica di Azienda Zero

Prima di approfondire le tematiche più inerenti allo sviluppo del software e ai relativi concetti, architetture e vincoli, descriveremo l'infrastruttura di Azienda Zero, di tipo hybrid cloud. I servizi basati su architettura a microservizi, o comunque orientati al cloud, sono erogati principalmente tramite Amazon Web Services (AWS).

Le principali tecnologie che ospitano le applicazioni sviluppate o acquisite da Azienda Zero sono le seguenti:

- ambienti virtuali VMware, supportati da un'infrastruttura Oracle RAC per le basi di dati
- IaaS e PaaS in AWS
- infrastruttura Kubernetes con EKS
- Atlas MongoDB

L'approccio primario di Azienda Zero si focalizza sull'adozione di microservizi utilizzando l'infrastruttura di Kubernetes e i servizi PaaS. Questo documento riflette chiaramente l'intenzione di orientare gli sviluppi in questa direzione metodologica e tecnologica.

1.1.1 Resilienza e Affidabilità

La struttura tecnologica di Azienda Zero è idonea per ospitare applicazioni resilienti e affidabili, grazie alle soluzioni presentate in questo documento. Nell'ambito dello sviluppo delle applicazioni, è essenziale integrare in modo nativo concetti di elevata disponibilità e continuità operativa, soprattutto adottando approcci moderni come microservizi e dominio dei dati, l'utilizzo di database distribuiti e la separazione degli strati applicativi.

È altresì importante far riferimento a fonti di letteratura specializzata e ai vincoli posti dalle normative nazionali ed europee relative alla digitalizzazione della Pubblica Amministrazione (PA).

1.2 Presa in carico di nuove applicazioni

La distribuzione delle applicazioni prodotte deve rispettare le policy definite da Azienda Zero le cui fasi sono di seguito elencate:

1. Consegna del codice
2. Analisi statica del codice
3. Analisi di sicurezza del codice
4. Compilazione del codice
5. Unit e integration test
6. Rilascio negli ambienti non di produzione
7. Test Automatici e di non regressione
8. Collaudo dell'applicativo
9. Rilascio in produzione

Lo stesso processo dev'essere rispettato per le successive distribuzioni, sia che si tratti di modifiche evolutive sia correttive.

Le opzioni descritte precedentemente non sono le uniche soluzioni disponibili, sebbene siano le preferite. Di seguito, illustriamo le alternative possibili. Tuttavia, è cruciale sottolineare che queste alternative saranno valutate come soluzioni di ultima istanza dettate da situazioni emergenziali e da considerare con minor preferenza.

Il codice sorgente sviluppato per conto di Azienda Zero sarà di proprietà di Azienda Zero e in quanto tale potrà essere fornito in riuso ad altre Pubbliche Amministrazioni.

1.2.1 Assegnazione di uno o più Account AWS

In relazione a quanto sopra esposto, e previa valutazione da parte di Azienda Zero, il fornitore ha la possibilità di utilizzare account AWS dedicati, gestiti dal proprio team di supporto tecnico specializzato interno, per le seguenti finalità:

- **Sviluppo e rilascio di applicazioni:** Questa opzione permette al fornitore di sviluppare e rilasciare applicazioni senza che venga attivata la fornitura effettiva del servizio. Gli

account sono assegnati per facilitare il rapido progresso nelle fasi iniziali di sviluppo e integrazione, per poi rientrare nel ciclo di vita delineato all'inizio di questo capitolo.

- **Sviluppo, rilascio ed effettiva erogazione del servizio:** In questo caso, il fornitore può sviluppare, rilasciare ed erogare effettivamente il servizio utilizzando solo l'infrastruttura di sicurezza di Azienda Zero. La responsabilità dell'erogazione effettiva del servizio ricade direttamente sul fornitore, supportato da Azienda Zero che offrirà e gestirà i servizi di sicurezza perimetrale.

La principale differenza tra le due opzioni risiede nella presenza o assenza dell'erogazione effettiva del servizio tramite gli account designati.

In ogni circostanza, il fornitore è tenuto a rispettare integralmente le disposizioni descritte nel presente documento e ad assicurare un trattamento adeguato e sicuro dei dati, in conformità con le normative vigenti.

I costi associati a questi account saranno coperti dai contratti stipulati con Azienda Zero, e il budget previsto all'inizio del progetto dovrà essere rigorosamente rispettato.

1.2.2 Presa in carico di Account AWS

Azienda Zero per esigenze di urgenza e opportunità può chiedere al fornitore di software anche la presa in carico dell'erogazione del servizio; in tal caso il servizio si configura come un vero e proprio SaaS.

Se il servizio è erogato da uno o più account AWS, una volta superato il periodo emergenziale Azienda Zero potrebbe riacquisire l'infrastruttura, importando tale account all'interno della propria organizzazione e quindi nel proprio contratto economico.

Anche in questo caso si chiede il rispetto delle prescrizioni del presente documento e la garanzia di un trattamento corretto e sicuro dei dati secondo la normativa vigente.

1.2.3 Consegna di software precompilato

Qualora il codice sorgente non sia di proprietà di Azienda Zero, dovrà essere consegnato il codice compilato e la relativa documentazione per dar seguito alle fasi descritte in precedenza.

2 CONCETTI DI BASE SULL'ARCHITETTURA A MICROSERVIZI

In questo capitolo vengono elencati i concetti fondamentali da applicare quando si sviluppa un'architettura a microservizi. Questi concetti sono esposti in modo conciso, e l'obiettivo della loro presentazione è quello di definire la Reference Architecture trattata nel documento. Per una trattazione più approfondita sull'argomento, è possibile fare riferimento alla letteratura specifica. Le informazioni discusse in questo capitolo costituiscono le basi su cui si fonda la definizione della soluzione proposta.

2.1 Individuazione dei microservizi

Un'architettura a microservizi rappresenta un approccio per la realizzazione del backend di un'applicazione, suddividendo l'intero sistema in piccoli servizi distinti. Ciascun servizio opera in modo autonomo e interagisce con altri processi attraverso protocolli come HTTP/HTTPS, WebSocket o sistemi di messaggistica come il modello Publish/Subscribe. Ogni microservizio incorpora un insieme coeso di funzionalità aziendali rientranti in un contesto definito e circoscritto noto come "Bounded Context".

Deve essere sviluppato in modo indipendente e con la capacità di essere implementato separatamente dagli altri. Inoltre, ogni microservizio assume la responsabilità esclusiva sia dei dati sia della logica di dominio a cui è associato, garantendo così una gestione decentralizzata e autonoma dei dati. La scelta delle tecnologie di archiviazione dati (SQL, NoSQL) e dei linguaggi di programmazione può variare tra i diversi microservizi.

Durante lo sviluppo di un microservizio, è cruciale creare legami flessibili tra i servizi al fine di garantire autonomia nello sviluppo, nella distribuzione e nella scalabilità di ciascun modulo. Nel processo di individuazione e progettazione dei microservizi, è auspicabile mirare a renderli il più compatti possibile, purché non ci sia un eccessivo numero di interdipendenze dirette con altri microservizi. Oltre alle dimensioni stesse, è fondamentale concentrarsi sulla coerenza interna che esso deve presentare e sulla sua capacità di operare in maniera indipendente rispetto agli altri servizi.

Per individuare i confini di un microservizio, è essenziale concentrarsi sul modello logico dei dati dell'applicazione e cercare di individuare gruppi isolati e non correlati di dati. Ogni insieme distinto di dati così identificato rappresenta un candidato promettente per costituire la parte dati di un "Bounded Context" (contesto circoscritto). Questi contesti dovrebbero essere definiti e gestiti in modo indipendente l'uno dall'altro. Ogni contesto potrebbe adottare un linguaggio di business diverso, con terminologie e entità uniche. Pur sembrando simili, i termini e le entità utilizzati in diversi contesti potrebbero assumere significati diversi a seconda del contesto in cui vengono impiegati. Per esempio, il termine "utente" potrebbe essere usato nell'ambito dell'autenticazione, mentre nello scenario funzionale potrebbe assumere le connotazioni di "paziente", e ancora diversamente come "medico" in un altro contesto.

2.2 Architettura logica e architettura fisica

Tutto ciò che è stato discusso in merito ai microservizi, in particolare alla loro stretta connessione con il concetto di "Bounded Context", contribuisce a fornire una definizione concettuale dei microservizi. Tuttavia, nella realtà, questa definizione concettuale potrebbe non coincidere direttamente con l'implementazione fisica effettiva dei componenti. Per esempio, quando si analizza il dominio, potrebbe emergere un contesto chiamato X, caratterizzato da un insieme di entità e una specifica logica funzionale. In certi casi, alcune di queste funzionalità potrebbero avere un carico di utilizzo più elevato rispetto ad altre. Di conseguenza, potrebbe essere una scelta valida creare due componenti fisici distinti: uno che gestisce le funzionalità ad alto utilizzo e un altro per quelle meno frequenti. Entrambi i componenti condividerebbero lo stesso database poiché fanno parte dello stesso "Bounded Context" (microservizio logico). Tuttavia, sfruttando questa separazione fisica dei microservizi, sarebbe possibile adottare strategie di distribuzione distinte per affrontare le variazioni di carico.

Il punto importante è che un microservizio logico deve essere autonomo, così da poter essere mantenuto indipendentemente dalle altre parti del sistema.

2.3 Sovranità dei dati per microservizio

Un principio fondamentale nell'architettura dei microservizi è che ciascun microservizio debba essere responsabile in modo esclusivo della gestione dei propri dati e della logica del dominio a cui si riferisce. Analogamente a come un'applicazione completa detiene la sua logica e i suoi dati, ogni microservizio dovrebbe possedere la sua logica e i suoi dati, evolvendo in un ciclo di vita autonomo. Questo approccio consente di distribuire i servizi in modo indipendente gli uni dagli altri.

Questo implica che il modello concettuale del dominio varierà da un microservizio all'altro. Un'entità che ha un significato concettuale nel dominio di un microservizio potrebbe avere un significato differente nel contesto di un altro microservizio. Questo principio è in linea con l'approccio del Domain Driven Design, in cui ogni Bounded Context, sottosistema o servizio autonomo deve possedere il proprio modello di dominio, comprensivo di dati, logica e comportamento. Il concetto di Bounded Context è strettamente legato al concetto di microservizio. Laddove il Bounded Context rappresenta un contesto logico, anche applicabile a sottosistemi di un sistema monolitico, il microservizio agisce come un Bounded Context distribuito, utilizzando i protocolli menzionati in precedenza, come HTTP/HTTPS, WebSocket o sistemi di messaggistica di tipo Publish/Subscribe, per stabilire comunicazione con le altre parti del sistema.

L'accesso ai dati diventa considerevolmente più complesso all'interno di un'architettura a microservizi. Sebbene sia possibile utilizzare le transazioni a livello di database all'interno di ciascun microservizio, i dati gestiti da ogni microservizio sono di natura privata e accessibili esclusivamente attraverso la sua API. Questo incapsulamento dei dati è fondamentale per garantire un debole accoppiamento tra i microservizi e la loro capacità di evolvere indipendentemente gli uni dagli altri. Se diversi servizi accedessero agli stessi dati, le modifiche allo schema richiederebbero aggiornamenti coordinati per tutti i servizi, compromettendo

l'autonomia del ciclo di vita dei microservizi. Tuttavia, il fatto di distribuire le strutture dati su diversi microservizi comporta la perdita del contesto transazionale. Di conseguenza, è necessario adottare altre strategie per mantenere la coerenza dei dati quando un'operazione di business si estende su più microservizi.

Inoltre, microservizi diversi utilizzano spesso diversi tipi di database. Le moderne applicazioni archiviano ed elaborano diversi tipi di dati e un database relazionale non è sempre la scelta migliore. Per alcuni casi d'uso, un database NoSQL potrebbe avere un modello dati più conveniente e offrire prestazioni e scalabilità migliori rispetto a un database SQL. In altri casi, un database relazionale è ancora l'approccio migliore. Pertanto, le applicazioni basate su microservizi utilizzano spesso una combinazione di database SQL e NoSQL.

2.3.1 Lettura di dati di competenza di più microservizi

In un'architettura a microservizi i dati sono distribuiti tra i vari microservizi che la compongono e non è immediato risolvere il problema di ottenere dati aggregati quando ve ne sia la necessità. Ciascun microservizio è proprietario dei dati che gestisce ed utilizza un proprio database, separato da quello degli altri microservizi, per gestire questi dati. È intuitivo comprendere che in uno scenario del genere non è possibile aggregare dati di diversi microservizi con una semplice join.

Schematizzando, si possono distinguere due principali situazioni in cui il problema si manifesta:

1. Aggregazione di dettaglio di dati eterogenei – è il caso in cui è necessario accedere a dati mantenuti da un certo numero di microservizi e si rischia di creare un elevato traffico tra client e microservizi per recuperare questi dati. Ad esempio, su una pagina del front end sono raggruppati dati che sono di proprietà di diversi microservizi e per recuperarli è necessario invocare uno ad uno tutti questi microservizi.
2. Reportistica di dati eterogenei – è il caso in cui è necessario produrre report o comunque uno scarico massivo di dati aggregati di proprietà di microservizi diversi.

Nel primo caso l'approccio suggerito è quello di utilizzare il pattern Backend For Frontend o BFF, in questo caso il compito di collezionare i dati provenienti dai diversi microservizi proprietari dei dati da estrarre per il client è delegato ad un componente di back end. In questo modo le chiamate ai singoli microservizi sono eseguite direttamente nell'infrastruttura e questo mitiga il peso in termini di prestazioni che le numerose chiamate avrebbero se effettuate direttamente dal client. Il Backend For Frontend, espone verso il client un'interfaccia già ritagliata per le esigenze di quest'ultimo. Questo è vero per client che non risiedono sul back end, tipo le Single Page Application, le Web Application che usano connessioni asincrone lato browser o le App Mobile. Diverso è il discorso per quelle Web Application che prevedono l'esistenza di un back end (Web Application Server Side) per la produzione delle pagine HTML, in questo caso il problema non si pone.

Nel secondo caso, la reportistica di dati eterogenei, è consigliabile utilizzare un approccio tipo CQRS (Command Query Responsibility Segregation) nel quale i dati gestiti nelle transazioni dei singoli microservizi vengono duplicati in altri database dedicati alle operazioni di lettura massiva. In tal senso i dati possono essere duplicati in modo denormalizzato così da essere già nel formato adatto alle estrazioni desiderate. Il meccanismo che consente la duplicazione dei dati può essere basato su un sistema di messaggistica ad eventi che permette di disaccoppiare le

operazioni in transazione dei microservizi con la duplicazione dei dati. Il sistema di messaggistica dovrà lavorare in modo quasi-sincrono per garantire di fatto l'accesso in tempo reale ai dati aggregati. Questo approccio non solo risolve il problema originale (come eseguire query e unire i vari microservizi) ma migliora anche notevolmente le prestazioni rispetto a un join complesso poiché nel database coi dati duplicati questi sono già presenti nella forma necessaria all'applicazione. Naturalmente questo approccio implica un ulteriore lavoro di sviluppo e sarà necessario contemplare il modo di ottenere la coerenza dei dati raggruppati per la reportistica.

2.3.2 Scrittura dati su più microservizi

Per quanto detto finora ciascun microservizio è proprietario dei dati che gestisce e non deve mai accadere che un microservizio accede direttamente in lettura o scrittura ai dati di un altro microservizio. Questo significa che se un'operazione di business ha la necessità di scrivere dati che appartengono a microservizi distinti è necessario che questi ultimi vengano invocati in maniera indipendente per scrivere ciascuno i dati di sua esclusiva competenza. È immediato verificare che in questo caso non è possibile utilizzare strategie di approccio ai dati che consentono di gestire transazioni fisiche sui database (ACID – Atomicity, Consistency, Isolation, Durability). In generale, per i sistemi distribuiti vale quanto affermato dal teorema CAP, ovvero che è impossibile che un archivio dati distribuito fornisca contemporaneamente più di due delle seguenti tre garanzie:

- Coerenza: Ogni lettura riceve la scrittura più recente o un errore
- Disponibilità: Ogni richiesta riceve una risposta (non-errore), senza la garanzia che contenga la scrittura più recente
- Tolleranza alle partizioni: Il sistema continua a funzionare nonostante un numero arbitrario di messaggi sia stato abbandonato (o ritardato) dalla rete tra i nodi

In un'architettura a microservizi la coerenza nella scrittura dei dati è realizzata in modo sincrono e asincrono. Nel caso sincrono i microservizi che partecipano alla scrittura sono attivati da un componente detto orchestratore (cfr. § [3.9](#)). Nel caso asincrono si utilizza un sistema di messaggistica ad eventi che permette di propagare le modifiche ai dati tra i vari microservizi coinvolti utilizzando i meccanismi di Publish/Subscribe. Se un'operazione di business richiede la scrittura di dati su più microservizi è necessario pubblicare l'evento di scrittura su cui si attiveranno i microservizi sottoscritti che provvederanno all'allineamento dei dati di propria competenza (cfr. § [2.3](#)). In entrambi i casi, sincrono o asincrono, è importante che prima di attivare la catena di scritture sui vari microservizi siano effettuati gli opportuni controlli logici che verifichino l'effettiva fattibilità dell'operazione su tutti i microservizi coinvolti.

2.4 Asincronia nell'architettura a microservizi

Per quanto detto al paragrafo precedente, in un'architettura a microservizi è necessario tener conto di un certo grado di asincronia nelle operazioni dell'utente. Questo significa che, in alcuni casi, a fronte di un'azione eseguita sul front end il sistema non risponde immediatamente; pur restituendo il controllo all'utente che lavora sul browser, non è detto che l'azione da questo avviata sia stata effettivamente portata a termine. Di questa asincronia deve essere tenuto conto

nelle fasi di sviluppo ed in particolare nella definizione dell'Interfaccia Utente. È importante che il committente sia consapevole del comportamento asincrono del sistema e di come, di conseguenza, potrebbe cambiare l'esperienza dell'utente nell'utilizzare l'applicazione.

Il problema dell'asincronia può essere affrontato in modo diverso a seconda del grado di asincronia della specifica operazione. Per le operazioni asincrone il cui tempo di esecuzione può essere garantito al di sotto di un certo lasso di tempo tale da far sembrare l'operazione sincrona, è il caso delle operazioni quasi-sincrone, non è necessario adottare particolari accorgimenti e l'interattività utente può essere assimilata a quella delle operazioni sincrone (il sistema aspetta che l'operazione sia completata e restituisce l'esito all'utente). Per le operazioni asincrone che richiedono un maggior tempo di esecuzione e non possono ricadere nella categoria delle operazioni quasi-sincrone, è necessario modificare l'interattività dell'utente prevedendo l'invio di messaggi opportuni che comunichino all'utente l'effettivo completamento dell'operazione. Tali messaggi possono essere gestiti con la tecnologia WebSocket (cfr. § [3.8](#)).

2.5 Comunicazione tra microservizi

Un'architettura a microservizi è un sistema distribuito ed in quanto tale richiede che la strategia di comunicazione fra le parti che lo costituiscono sia progettata accuratamente. Per ottenere un sistema che trae il massimo dei vantaggi da questa tipologia di architettura non è sufficiente limitarsi alla scelta dei protocolli di comunicazione (HTTP, REST, sistemi di messaggistica tipo Publish/Subscribe e così via). Due parti del sistema che comunicano tra loro sono di fatto correlate l'una con l'altra. La comunicazione instaura un legame che in qualche modo definisce un accoppiamento tra i microservizi e questo accoppiamento deve essere sempre considerato nella progettazione di questo tipo di architetture. In un'architettura a microservizi è frequente la scelta di utilizzare come protocollo di comunicazione l'HTTP. Di per sé la scelta di questo standard per le comunicazioni è una scelta plausibile, ma, se il partizionamento in microservizi individuato comporta un gran numero di connessioni tra questi, per assurdo, si potrebbero generare tanti legami da ricondurre l'intera architettura ad un sistema monolitico le cui comunicazioni sono basate su HTTP. Per stabilire una corretta strategia di comunicazione è necessario tenere presenti quali sono i problemi che un approccio sbagliato o non pianificato può determinare.

- Basse prestazioni. A causa della natura sincrona dell'HTTP, la richiesta originale non riceve risposta fino al termine di tutte le chiamate HTTP interne. Se il numero di queste chiamate aumenta in modo significativo il risultato è che le prestazioni peggiorano e la scalabilità generale sarà influenzata in modo incontrollato all'aumentare delle richieste HTTP aggiuntive in una sorta di effetto valanga.
- Scarsa resilienza. In una catena di microservizi collegati da chiamate HTTP, quando uno qualsiasi dei microservizi fallisce, l'intera catena di microservizi fallisce. Un sistema basato su microservizi dovrebbe essere progettato per continuare a funzionare nel miglior modo possibile durante i guasti parziali.
- Accoppiamento. L'eccesso di comunicazione tende ad accoppiare i microservizi. Idealmente un microservizio non dovrebbe avere legami con gli altri microservizi che

costituiscono l'architettura. Più sono i legami con gli altri microservizi più difficilmente si realizza l'autonomia nella gestione del singolo componente.

In generale, la necessità di creare tante connessioni per un microservizio potrebbe dipendere da un'errata definizione del Bounded Context individuato in fase di progettazione. Se, ad esempio, tra due microservizi si individuano molti punti di connessione, ovvero di interdipendenza, bisogna prendere in considerazione la possibilità di unificarli.

Per quanto detto, al fine di ottenere l'autonomia dei microservizi, avere una migliore resilienza e migliori prestazioni è necessario ridurre al minimo l'uso di catene di comunicazione richiesta/risposta tra microservizi.

2.6 ROA e SOA

Con gli acronimi ROA e SOA, rispettivamente Resource Oriented Architecture e Service Oriented Architecture, si fa riferimento a due possibili modalità di progettazione delle funzionalità esposte dal back end.

Col paradigma ROA le API del back end rendono le risorse, ovvero le entità di dominio, disponibili ad un uso distribuito. Sulle risorse è possibile effettuare le operazioni di lettura, creazione, aggiornamento e cancellazione tipiche del protocollo HTTP (GET, POST, PUT, DELETE, ecc.). La progettazione ROA abilita la creazione di API (Application Programming Interface) RESTful e facilita l'adozione di strategie di caching (cfr. § [2.6.1](#)).

Col paradigma SOA le API del back end espongono servizi, ovvero operazioni, generalmente associate ai metodi HTTP GET e POST.

Per il dettaglio sui due paradigmi di progettazione si rimanda alla letteratura di riferimento. In questa sede si vuole rimarcare la differenza tra i due paradigmi e vincolare il progettista a distinguere nettamente i servizi da realizzare in modo da scegliere l'approccio giusto a seconda del caso. Inoltre, ove possibile, si invita a preferire il paradigma ROA per ottenere i vantaggi di un approccio RESTful e di caching detti in precedenza. Resta scontato che la scelta specifica di come esporre i servizi di back end è legata alle particolari esigenze applicative.

A titolo di esempio si consideri il microservizio che ha nome "pratiche" ed il cui dominio comprende la gestione dell'entità PRATICA. Si supponga che le funzionalità del dominio comprendano:

- lettura pratiche – fornisce una lista delle pratiche inserite ed offre la possibilità di filtrare il risultato in base ad alcuni attributi dell'entità PRATICA (es. data creazione, nominativo richiedente, stato di lavorazione, ecc.)
- dettaglio pratica – fornisce il dettaglio della pratica
- creazione pratica – permette la creazione di una pratica
- modifica pratica – permette la modifica di una pratica
- inoltro pratica – esegue l'operazione di inoltro pratica all'ufficio destinatario

Per tutte le funzionalità elencate eccetto quella di inoltro pratica sarebbe possibile adottare un approccio ROA esponendo le funzionalità sull'URI (Uniform Resource Identifier) /pratiche ed utilizzando di volta in volta il metodo HTTP corrispondente (GET per le letture, POST per la creazione, PUT per la modifica). Per la funzionalità di inoltro pratica si potrebbe adottare il

paradigma SOA ed utilizzare l'URI /pratiche/[id pratica]/inoltro, dove [id pratica] è l'identificativo univoco della pratica ottenuto dalle operazioni di lettura o di creazione.

2.6.1 Caching in REST

Il concetto di caching è centrale nel design di una architettura REST, anche se non tutti i verbi supportano il caching:

- GET: cacheable by design
- POST: non cacheable by design ma si può ovviare a ciò utilizzando degli specifici header HTTP.
- PUT: non cacheable
- DELETE: non cacheable

Il meccanismo di caching è gestito mediante l'utilizzo di una serie di header HTTP, attraverso i quali viene ottimizzata l'acquisizione dello stato della risorsa e viene gestito il salvataggio della stessa evitando conflitti.

3 ARCHITETTURA APPLICATIVA DI RIFERIMENTO

In base a quanto detto finora, l'architettura applicativa di riferimento individuata per una Web Application è quella rappresentata negli schemi di Figura 1 e Figura 2; rispettivamente nel caso si utilizzi come front end una Single Page Application (SPA) o una Web Application Server Side.

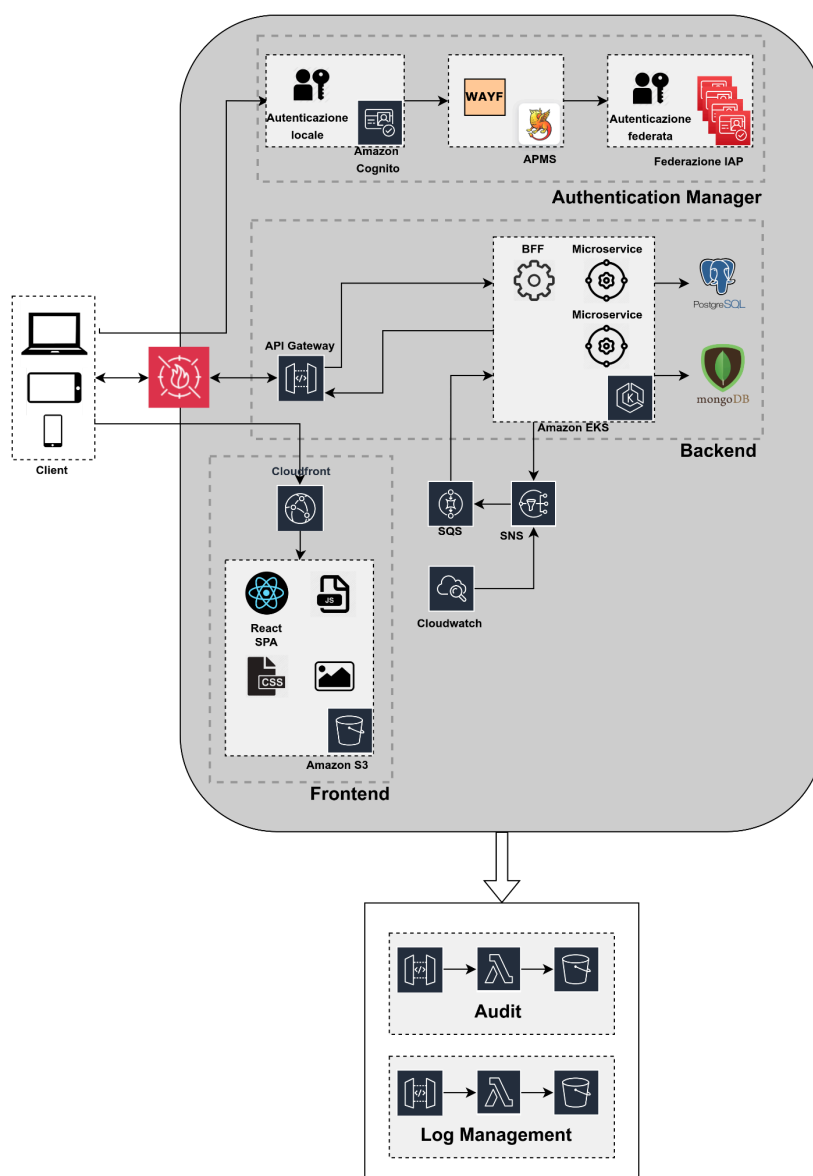


Figura 1 - Architettura applicativa di riferimento per SPA

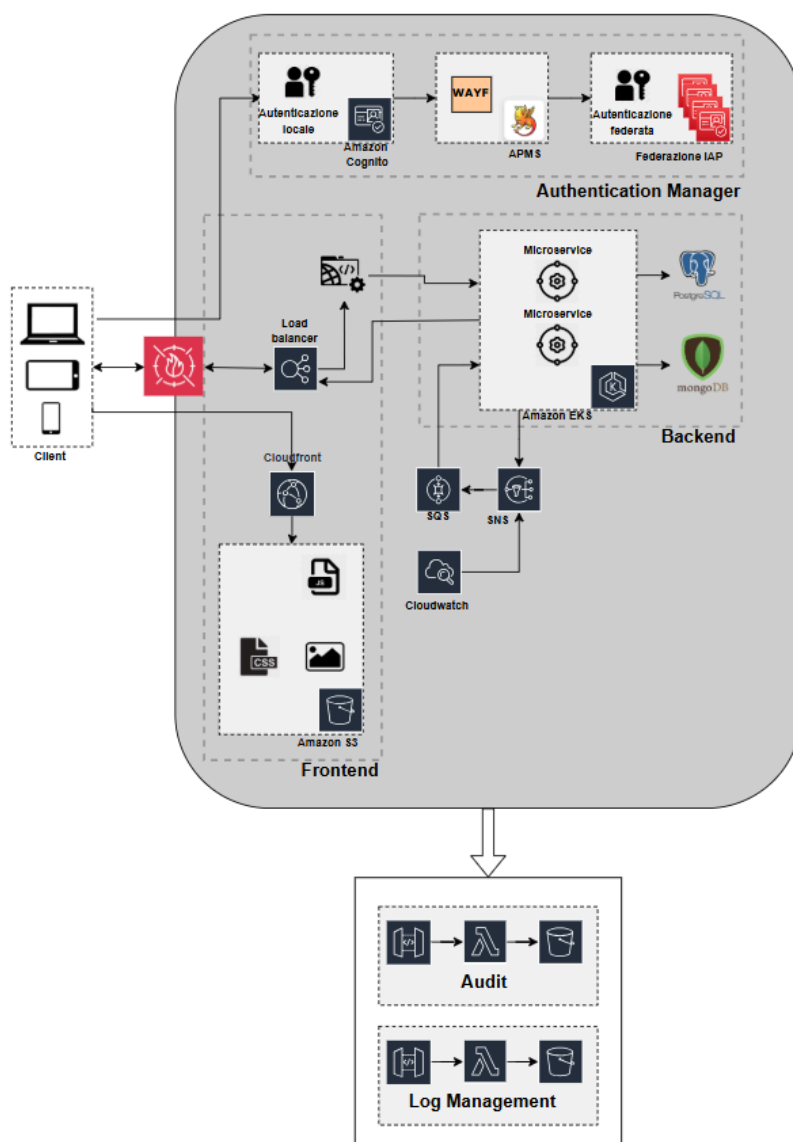


Figura 2 - Architettura applicativa di riferimento per Web App Server Side

I due schemi sono molto simili e si differenziano esclusivamente in riferimento alla diversa tecnologia utilizzata per realizzare il front end.

Nello schema sono evidenziati cinque macro blocchi che fanno riferimento al Front end, al Back end, all'Authentication Manager, all'Operation Log e all'Audit GDPR.

3.1 Front end

Per il front end si possono utilizzare sia Single Page Application sia Web Application Server Side in base alle esigenze specifiche di progetto. Nel caso di SPA, la tecnologia da utilizzare è ReactJS, Angular2 o VueJS.

Una Single Page Application (SPA) è un tipo di applicazione web che comunica direttamente con il browser web, modificando in tempo reale la pagina web attuale in base ai dati scambiati con il server anziché ricaricare interamente la pagina da zero ogni volta che si richiede una nuova visualizzazione. Lo scopo principale di una SPA è ottenere transizioni più fluide e veloci, dando al sito web un aspetto simile a un'applicazione mobile nativa.

In una SPA, tutto il codice HTML, JavaScript e CSS necessario viene caricato una sola volta quando si accede alla pagina iniziale o viene recuperato dinamicamente in base alle esigenze, solitamente in risposta alle azioni dell'utente. Questo significa che la pagina non viene mai completamente ricaricata durante l'utilizzo dell'applicazione e non si passa a nuove pagine. Tuttavia, è possibile sfruttare l'hash della posizione o l'API History HTML5 per dare l'impressione di navigare tra pagine separate all'interno dell'applicazione.

Il termine "Web Application Server Side" fa riferimento a un tipo di applicazione web considerata finora come "tradizionale". In questo contesto, sia le pagine visualizzate nel browser dell'utente sia la logica di gestione delle interazioni con l'utente sono gestite sul lato del server..

3.2 Back end

3.2.1 API Gateway e API Management

Nel caso di front end di tipo SPA le risorse di back end sono esposte tramite API Gateway sul quale sono definite le rotte per l'accesso alle risorse dei microservizi. L'API Gateway va programmato per esporre l'interfaccia voluta. Sull'API Gateway devono essere definiti i criteri autorizzativi che permettono di controllare gli accessi verso i microservizi.

Il controllo di autorizzazione all'accesso alle risorse di back end è realizzato tramite JWT (JSON Web Token). In particolare, si utilizza l'ID_Token del protocollo Open ID Connect (OIDC). L'esistenza dell'ID_Token, la sua integrità e le risorse per le quali è valido sono controllate dalla funzione Lambda Authorizer montata sull'API Gateway. La Lambda Authorizer verifica anche che il ruolo applicativo dell'utente, anch'esso aggiunto nell'ID_Token dall'Authentication Manager, sia tra i ruoli autorizzati ad accedere alle risorse richieste. Un accesso che non verifica le condizioni controllate da Lambda Authorizer è rifiutato con Access Denied. Il flusso di autenticazione/autorizzazione è descritto in dettaglio al capitolo 5.

Sia per front end di tipo SPA, che per front end Web App Server Side, l'API Gateway può essere utilizzato per la gestione delle connessioni WebSocket per le comunicazioni verso il browser (cfr. § 3.8).

La Lambda Authorizer è realizzata in NodeJS per ragioni inerenti alle prestazioni dell'applicazione. Il componente è installato come AWS Lambda Function.

Nel caso il servizio preveda di esporre API ad altre applicazioni le stesse verranno integrate e gestite nel sistema di API Management.

3.2.2 Microservizi

Sul back end risiedono i microservizi che costituiscono l'applicazione lato server. Ciascuno di essi è collegato al proprio database per realizzare la sovranità dei dati di cui si è discusso al § [2.3](#).

Sebbene tra le prerogative di un'architettura a microservizi ci sia quella di poter valutare di caso in caso il tipo di tecnologia da utilizzare per realizzare un microservizio, se non ci sono controindicazioni particolari, i microservizi sono realizzati in Java ed i DB di riferimento sono PostgreSQL e MongoDB.

I microservizi sono incapsulati in container che a loro volta sono aggregati in gruppi da uno o più container in POD.

I microservizi sono installati nei clusters Kubernetes come POD o in alternativa su Lambda. I database sono esterni al pod e, ove possibile, soluzioni PaaS del cloud provider.

3.2.3 Backend For Frontend (BFF)

Il componente BFF, ovvero Backend For Frontend, presente solo nel caso si decida di utilizzare una SPA, è adoperato per migliorare le strategie di accesso ai dati quando sono richiesti, in un'unica operazione, dati proprietari di più microservizi (cfr. § [2.3.1](#)). Il BFF è utilizzato quando la richiesta di dati a più microservizi diventa un fattore rilevante in termini di prestazioni dell'applicazione. Il BFF ricalca il pattern Façade e di fatto si comporta come trasduttore della logica delle operazioni esposte dai microservizi nella logica a servizi richiesta dalle operazioni del front end. Mentre i microservizi sono realizzati tenendo presente il dominio applicativo, il BFF è strettamente accoppiato all'Interfaccia Utente ed è realizzato considerando i dati da estrarre nel corso della navigazione.

Il BFF, lì dove necessario, può anche gestire le connessioni WebSocket per le comunicazioni verso la SPA tramite l'API Gateway (cfr. § [3.2.1](#)).

Il BFF è realizzato in uno dei linguaggi supportati da Azienda Zero. Il componente è installato nel cluster Kubernetes come immagine Docker.

3.3 Mobile Application

In caso di realizzazione di applicativi mobile possono essere utilizzate le seguenti tecnologie:

- Flutter per realizzazione di app cross platform. Flutter è un framework open-source creato da Google per la creazione di interfacce native per Android e iOS.
- React Native per realizzazione di app cross platform. React Native è un framework per applicazioni mobile open-source creato da Meta. Viene utilizzato per sviluppare applicazioni per Android e iOS.
- Kotlin per la realizzazione di app native Android
- Objective-C and Swift per la realizzazione di app native IOS

Indipendentemente dalle tecnologie scelte, la memorizzazione e l'accesso a eventuali dati personali deve avvenire garantendo adeguati livelli di sicurezza. Si riportano, a titolo esemplificativo, ma non esaustivo, alcune best practice:

- cifratura dei dati in-transit
- cifratura dei dati at-rest
- isolamento dei dati dalle altre applicazioni presenti sul medesimo dispositivo
- l'applicazione, in fase di installazione e di avvio, deve verificare se il dispositivo abbia mantenuto i livelli di sicurezza (quindi non sia rooted, jailbroken, ecc.), anche tramite l'utilizzo delle API offerte dal produttore
- se i requisiti indicati precedentemente non vengono soddisfatti, è necessario informare l'utente affinché abbia la possibilità di interrompere l'installazione e/o l'utilizzo dell'applicativo; altrimenti, l'utente può scegliere di proseguire assumendosene la responsabilità

Inoltre:

- non devono essere inseriti dati personali nei log del dispositivo
- il debug non deve essere attivo per app di produzione
- la gestione degli errori ed eccezioni deve seguire quanto descritto nel § [5.7 Gestione degli errori](#)

3.4 Soluzioni CMS

Per esigenze di pubblicazione di siti e minisiti con soluzioni di tipo CMS ove i contenuti devono essere delegati ad utenti non tecnici si suggerisce l'uso dell'infrastruttura che ad oggi supporta il sito istituzionale di Azienda Zero (www.azero.veneto.it), il sito istituzionale della Salute di Regione del Veneto (salute.regione.veneto.it) e il sito delle relazioni socio sanitarie della Regione del Veneto (relazionesanitaria.azero.veneto.it).

3.5 Authentication Manager

È il sistema che permette all'utente di autenticarsi. È composto da tre componenti principali:

- Amazon Cognito – specifico per ciascuna applicazione abilita l'interfaccia OIDC (Open ID Connect) per il flusso di autenticazione. Per esigenze specifiche è possibile definire gli utenti applicativi anche direttamente su Cognito. Il processo di autenticazione produce un Token Autorizzativo che deve essere utilizzato per la gestione delle autorizzazioni all'accesso ai microservizi di back end.
- APMS – Sistema di profilazione applicativo. Associa all'utente le informazioni del profilo applicativo e del ruolo organizzativo. APMS si occupa anche dell'integrazione tra Cognito e la Federazione IAP tramite il prodotto Shibboleth configurato come IdPProxy che espleta le funzioni di WAYF (Where Are You From).
- Federazione IAP – Federazione degli Identity Provider tra i quali l'utente sceglie quello su cui autenticarsi.

Per ciascuna applicazione realizzata in ambiente AWS deve essere definito uno User Pool di Cognito per la gestione degli utenti. Il flusso di autenticazione degli utenti definiti direttamente su Cognito non attiva gli IdP della Federazione IAP.

Le modalità di integrazione col sistema di autenticazione sono spiegate con maggior dettaglio al § 4.

3.6 Gestione Eventi/Messaggi

Nei diagrammi di Architettura Applicativa sono evidenziati anche i servizi SNS ed SQS di AWS. Questi devono essere utilizzati per la gestione degli Eventi/Messaggi di tipo Publish/Subscribe da utilizzarsi ogni volta che è necessaria la gestione di task asincroni nell'ambito del processo.

3.7 Log Management

L'applicazione deve prevedere una gestione dei logs, in particolare è richiesta una distinzione netta dei logs prodotti per le seguenti finalità:

- Audit
- Analytics
- Debug

Per le modalità di integrazione con i sistemi di log management si rimanda alla documentazione di riferimento.

3.7.1 Audit

I logs di tipo audit contengono le informazioni relative agli accessi ed alle operazioni effettuate dagli utenti e da applicazioni esterne sui dati trattati. Con audit si definisce il tracciamento di una qualsiasi azione che un utente compie a livello applicativo.

La destinazione dell'audit è un metodo API esposto dall'infrastruttura di Azienda Zero che ne permette la scrittura del record. Il payload da utilizzare per l'invio del record tramite API deve essere in formato JSON e deve contenere i campi descritti di seguito.

Il record dovrà contenere i seguenti campi rispettando l'ordine seguente:

- **Timestamp**
 Data e Ora dell'evento
 Utilizzare lo standard RFC 9557 con la seguente notazione UTC %Y-%M-%DT%h:%m:%sZ (es. 2025-11-25T15:30:55Z)
- **Nome utente o codice applicativo chiamante**
 Identità dell'utente/applicazione che ha generato l'evento
 Un codice che permetta di determinare in modo assoluto l'utente o l'applicazione esterna che ha generato l'evento
- **Codice sessione**
 Identificativo della sessione che collega questa operazione alle precedenti
 Può essere utilizzato un codice come il cookie di sessione o altro che permetta la

tracciatura della sequenza delle azioni eseguite da un utente o altro applicativo in modo univoco.

- **Operazione**

Identificativo dell'operazione eseguita

Possono essere operazioni tipo *Accesso*, *Cancellazione*, *Modifica*, etc. o Operazioni specifiche dell'applicazione.

- **Esito**

Esito dell'azione intrapresa

Successo/Non Successo o altro che permetta di chiarire l'esito dell'evento

- **Applicazione**

Identificativo dell'applicazione

Può essere identificata anche una distinzione di moduli applicativi, utilizzare, comunque, il codice applicativo assegnato in MI (AP, Abbreviazione Progetto), meglio se recuperato da un parametro esterno all'applicazione.

Il record potrà contenere ulteriori informazioni sempre all'interno del messaggio json, se ritenute interessanti ai fini dell'audit dell'operazione. Queste informazioni devono essere contenute per evitare di rallentare l'operazione di salvataggio dei dati.

Il sistema di raccolta si basa sui servizi AWS. Le modalità di integrazione con il servizio sono descritte nel documento di specifica di integrazione del Log Management. I log saranno consultabili da Azienda Zero tramite gli strumenti di dashboard messi a disposizione dal servizio di Log Management, in base alle specifiche esigenze.

3.7.2 Analytics

I logs di tipo analytics contengono informazioni sull'andamento di un'applicazione. Questa categoria comprende dati come la durata e l'esito di operazioni significative all'interno dell'applicazione, che vengono registrati per scopi di analisi.

Il sistema di logging dedicato a questa categoria già raccoglie automaticamente i dati relativi alle visite direttamente dai dispositivi di bilanciamento e sicurezza. Pertanto, per queste informazioni di base, non è necessario inviare registri separati, a meno che non sia necessario arricchirli con ulteriori dettagli.

Il sistema di gestione dei logs analitici consente di creare dashboard per l'analisi delle tendenze o per la generazione di report aziendali sull'utilizzo e sulla qualità del servizio.

Il sistema utilizzato per la raccolta di questi log è il servizio centralizzato di Azienda Zero denominato Log Management, per l'integrazione si rimanda alla documentazione di riferimento.

3.7.3 Debug

Il log di tipo debug sono informazioni utili all'analisi del comportamento dell'applicativo e dei suoi componenti con finalità di correzione di errori o di miglioramento.

Il sistema di raccolta di questi log è AWS Cloudwatch. I log generati dall'applicativo possono essere inoltrati a Cloudwatch nelle seguenti modalità:

1. uso delle api/sdk di cloudwatch per l'invio dei record di debug.
2. Scrittura di file a file system (solo per sistemi basati su VM o EC2)
3. Scrittura in standard output per sistemi containerizzati

Nel caso di scrittura di file, è necessario diversificare gli stessi per tipologia di log. Sarà l'IT Operation, tramite agenti, a occuparsi della loro raccolta e invio al sistema centralizzato.

I sistemi containerizzati su Kubernetes raccolgono già i log inviati in standard output direttamente su CloudWatch.

3.8 Messaggistica WebSocket

Come menzionato in precedenza (cfr. § 3.6), in un'architettura a microservizi, può essere necessario creare un canale di comunicazione specifico per gestire le notifiche relative al completamento effettivo delle operazioni asincrone eseguite dal sistema. La tecnologia attualmente utilizzata per implementare questa forma di comunicazione è WebSocket.

In questo paragrafo, viene fornito un esempio di come implementare un canale di comunicazione WebSocket. L'obiettivo è offrire agli sviluppatori una guida su come integrare WebSocket nell'architettura a microservizi in questione. Si presta particolare attenzione al meccanismo di autorizzazione da utilizzare per consentire al client di aprire il canale, poiché questo rappresenta un requisito fondamentale nell'effettiva implementazione di WebSocket.

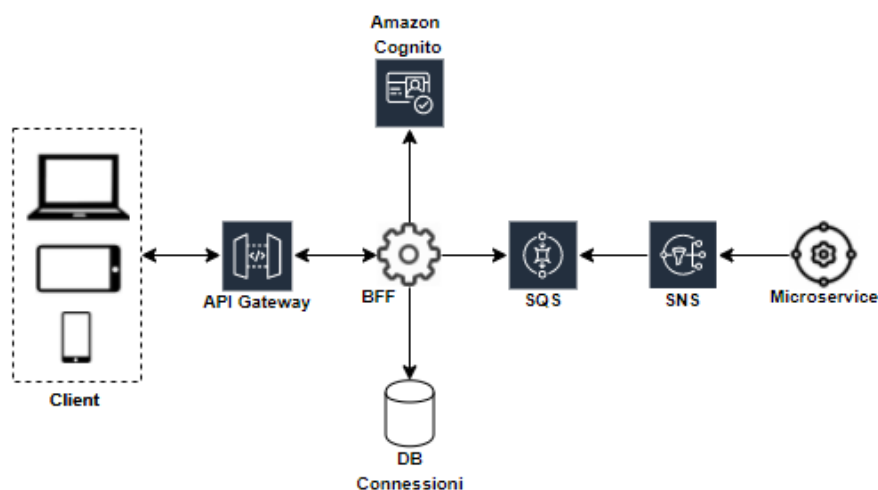


Figura 3 - Esempio implementazione WebSocket

Nell'esempio di Figura 3 è stato utilizzato AWS API Gateway per la gestione delle connessioni. Il gateway di AWS espone delle specifiche funzionalità che gli consentono di intercettare ed instradare le comunicazioni WebSocket sia dal back end verso il client sia viceversa.

Al momento dell'autenticazione, contestualmente al recupero dell'ID_Token, al client viene fornito un ticket di autenticazione, WS_Ticket, da spendersi per aprire la connessione WebSocket.

WS_Ticket persiste in un servizio di caching (es. Redis) ed è associato all'utente connesso. WS_Ticket ha un timestamp di scadenza oltre il quale non è più utilizzabile. La gestione di WS_Ticket non è implementata nativamente dall'API Gateway e, nell'esempio, è realizzata dal BFF.

Alla richiesta di connessione WebSocket, il client invia WS_Ticket come parametro in querystring. Se il WS_Ticket è ancora valido, la connessione viene stabilita e l'associazione utente-connectionId persiste in un servizio di caching (es. Redis). ConnectionId è l'id della connessione WebSocket. L'associazione utente-connectionId è cancellata al momento della disconnessione del client. Nell'esempio, la gestione delle connessioni è realizzata dal BFF.

BFF è sottoscritto tramite coda SQS a tutti gli eventi generati dai microservizi che rappresentano messaggi da comunicare al client, tipicamente messaggi di completamento di operazioni asincrone ma, con questo sistema, sono abilitati tutti i tipi di messaggi. Alla lettura dei messaggi sulla coda, BFF verifica se per l'utente a cui è destinato il messaggio esiste su DB Connessioni almeno un'associazione utente-connectionId e per ciascuna connessione WebSocket invia ai client corrispondenti il messaggio ricevuto sulla coda SQS.

3.9 Coreografia/Orchestrazione

Per quanto detto al § [2.3.2](#), ci sono casi in cui un'operazione di front end prevede la scrittura di dati che appartengono a più microservizi. L'attivazione in scrittura su più microservizi può seguire due possibili strategie:

- Coreografia - in questo caso, per mantenere la consistenza dei dati gestiti dai diversi microservizi si utilizza un sistema di messaggistica che propaga l'evento di scrittura dati da un microservizio o dal BFF o dalla Web App Server Side agli altri microservizi coinvolti. Si innesca così una coreografia che vede i microservizi attivarsi in scrittura alla ricezione dell'evento pubblicato dal microservizio o dal BFF o dalla Web App Server Side che ha intercettato l'operazione utente originaria.
- Orchestrazione - in questo caso la consistenza è garantita da un componente software, l'orchestratore appunto, che implementa il flusso delle chiamate ai vari microservizi coinvolti nell'operazione preoccupandosi di gestire anche eventuali fallimenti nelle chiamate mettendo in atto meccanismi di compensazione. Nell'architettura riportata in Figura 1 e Figura 2 il ruolo di orchestratore può essere svolto dal BFF o dalla Web App Server Side.

La scelta della strategia da adottare dipende dal caso specifico e va valutata di volta in volta.

In entrambi i casi è necessario verificare l'effettiva fattibilità dell'operazione prima di attivare la scrittura sulla catena di microservizi, ovvero, è necessario eseguire prima tutti i controlli formali e logici che potrebbero precludere l'esito dell'operazione.

In generale i sistemi di messaggistica non garantiscono che un messaggio sia consegnato una sola volta, può capitare che, a fronte di una pubblicazione, l'evento sia recapitato più volta alla funzione sottoscritta (in termini di Publish/Subscribe all'event sink). Per questa ragione è necessario che le componenti sottoscritte siano idempotenti.

Un problema che si pone nell'adottare la coreografia come strategia di scrittura è quello della concomitanza in un microservizio dell'azione di scrittura dati e della pubblicazione dell'evento sul

sistema di messaggistica. Per garantire che l'attivazione della catena di microservizi non si interrompa e quindi garantire la consistenza dei dati bisogna essere certi che, per un dato microservizio, le operazioni di scrittura dati e pubblicazione evento siano entrambe eseguite. Una possibile soluzione a questo problema è l'implementazione del Transactional Outbox Pattern spiegato di seguito.

3.9.1 Transactional Outbox Pattern

L'utilizzo di questo pattern risulta efficace nei casi in cui un microservizio deve aggiornare il database e contestualmente inviare messaggi/eventi. Ad esempio, un servizio che partecipa a una coreografia deve aggiornare atomicamente il database e pubblicare l'evento di scrittura sul sistema di messaggistica per attivare gli altri microservizi che partecipano alla coreografia. L'aggiornamento del database e l'invio del messaggio devono essere atomici per evitare incoerenze e bug nei dati. Tuttavia, non è possibile utilizzare una transazione distribuita che copre il database e il broker dei messaggi per aggiornare atomicamente il database e pubblicare messaggi/eventi.

Il pattern prevede che, invece di pubblicare il messaggio/evento sul sistema di messaggistica, un microservizio che utilizza un database relazionale inserisca i messaggi/eventi in una tabella di posta in uscita come parte della transazione locale. Nel caso di microservizio che utilizza un database NoSQL i messaggi/eventi vanno aggiunti come attributo al record (ad esempio documento o elemento) che è in fase di scrittura. Un processo separato di Message Relay pubblica gli eventi inseriti nel database sul broker di messaggi.

Uno schema d'esempio del pattern è riportato in Figura 4.

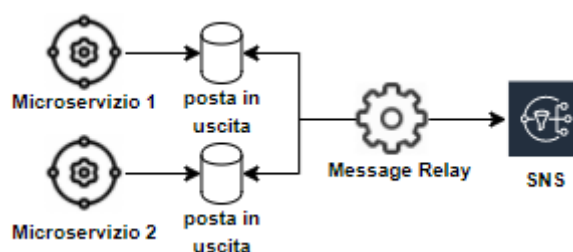


Figura 4 - Schema per Transactional Outbox Pattern

3.10 Schedulazione

Le schedulazioni dei microservizi su base temporale deve essere gestita tramite richiesta ad IT Operation di una schedulazione a livello Cronjobs di Kubernetes. Ogni cronjob avvia POD creati ad hoc per le funzioni schedulate.

Per approfondimento [Running Automated Tasks with a CronJob](#)

3.11 Comunicazioni Telefoniche

Il servizio COCE di Azienda Zero, basato su Amazon Connect, gestisce le chiamate in entrata e in uscita, rendendolo adatto a comunicazioni telefoniche con operatori o utenti. Per l'invio di SMS, Azienda Zero si avvale del servizio SaaS MailUP. L'utilizzo di questi o altri servizi richiede autorizzazione preventiva.

4 SISTEMI DI AUTENTICAZIONE

Azienda Zero fornisce alcune opzioni per verificare l'identità degli utenti. Si sconsigliano fortemente le metodologie di autenticazione che non sono centralizzate, e sono proibite quelle che non rispettano i requisiti minimi di sicurezza delle password o non permettono l'attivazione di soluzioni di autenticazione multifattore (MFA).

L'uso di un determinato sistema di autenticazione è preferibile o obbligatorio in base all'utenza dell'applicazione. Nelle tabelle seguenti sono riportate in modo conciso le diverse opzioni di autenticazione, l'utente a cui sono destinate e, se del caso, i requisiti o le informazioni importanti ad esse associate.

Azienda Zero offre l'ecosistema *APMS* come soluzione per l'integrazione con vari sistemi quali SPID, CIE, IAP, etc. Inoltre tale soluzione offre anche la possibilità di gestire una profilazione di tipo delegata.

Sistema di Autenticazione	Utenza	Vincoli e note
APMS (Application/Authentication Profile Management System)	Cittadini via SPID e CIE Utenti Censiti dalle Aziende Sanitarie Utenti Censiti da Regione del Veneto Utenti Censiti da Azienda Zero M2M (Machine to Machine)	Soluzione preferenziale, con supporto esclusivo per OpenID Connect. Importante: la soluzione offre sia autenticazione, sia profilazione. La soluzione permette di autenticare applicazioni per transazioni di backend.

Di seguito si elencano altri sistemi di autenticazione supportati da Azienda Zero.

Sistema di Autenticazione	Utenza	Vincoli e note
MyID	Cittadini via SPID e CIE	Soluzione alternativa, con supporto esclusivo per Saml2.
Google Cloud Identity Google Accounts	Dipendenti di Azienda Zero Fornitori di Azienda Zero Dipendenti delle Aziende Sanitarie	Soluzione preferibile rispetto a Active Directory/LDAP
Active Directory/Ldap WIN	IT Operations Fornitori di Azienda Zero Dipendenti di Azienda Zero	Soluzione legacy presente solo per la sua vasta compatibilità con prodotti

Sistema di Autenticazione	Utenza	Vincoli e note
	Utenti delle Aziende Sanitarie (Qlikview)	opensource e pacchetti software.
Active Directory VENEZIA	Postazioni di Lavoro di Azienda Zero	Soluzione legacy presente solo per situazioni particolari in cui l'applicazione necessita di integrarsi con il sistema di autenticazione utilizzato sulle postazioni di lavoro dei dipendenti di Azienda Zero
AWS SSO	IT Operations Fornitori di Azienda Zero	Ad uso delle sole applicazioni integrate con AWS SSO e solo per soluzioni infrastrutturali

4.1 Integrazione col sistema di autenticazione APMS

Ciascuna applicazione accede ai propri servizi di back end utilizzando un Token Autorizzativo JWT ottenuto da Cognito. Per ottenere il Token Autorizzativo l'applicazione deve avviare l'Authorization Code Flow del protocollo OIDC (OpenID Connect) esposto da AWS Cognito. È durante questa fase che scatta il flusso di autenticazione.

Il flusso di autenticazione, così come la sua implementazione, cambia in base alla tipologia degli utenti che devono poter accedere al sistema. Per una più chiara esposizione, gli utenti sono suddivisi in:

- Utenti Federati con SPID, CIE: Utenti che hanno aderito al circuito SPID o CIE
- Utenti Federati IAP: Sono utenti profilati nel circuito IAP
- Utenti Federati con Altri IDP: Cognito permette di federare altri IDP, tale opzione è da presentare preventivamente ad Azienda Zero per una valutazione.
- Applicazioni Autorizzate e profilate su APMS in fase di rilascio.

4.1.1 Componente di profilazione di APMS

L'implementazione di una nuova applicazione, generalmente, comporta la necessità di definire i Profili Applicativi dei Gruppi (tipologie) di utenti che vi possono accedere (es. Amministratore, Lettore, ecc.) e di una serie di attributi che caratterizzano nel dettaglio l'utente. In sintesi, il Profilo Applicativo è costituito da un insieme di informazioni associate all'utente che l'applicazione utilizza per realizzare nel dettaglio il RBAC (Role Based Access Control). Il Profilo Applicativo è gestito centralmente nel sistema APMS e per una esposizione più dettagliata si rimanda alla documentazione di riferimento.

Al termine del flusso di autenticazione il Token Autorizzativo conterrà l'attributo *apms:gruppi* contenente i gruppi ai quali appartiene l'utente separati da ",". Nel Token Autorizzativo saranno poi presenti gli attributi associati all'utente sul sistema APMS i cui nomi hanno il formato *apms:[nome attributo]* dove [nome attributo] è il nome dell'attributo definito in APMS.

4.1.2 Registrazione WAYF

Nel caso in cui nell'implementazione di una nuova applicazione è previsto l'accesso di Utenti Federati, allora è necessario effettuare la registrazione dello User Pool definito per l'applicazione come Service Provider del WAYF. Per la registrazione è necessario esporre i metadata dello User Pool su uno specifico endpoint. Questa operazione è effettuata sul sistema APMS e per una esposizione più dettagliata si rimanda alla documentazione di riferimento.

Una volta esposti, i metadata devono essere registrati nel WAYF (Shibboleth IdPProxy). Anche in questo caso si rimanda alla documentazione di riferimento.

4.1.3 Token Autorizzativo (ID_Token)

Nel riquadro di seguito un esempio reale di Token Autorizzativo per un Utente Federato.


```
{
  "at_hash": "pqU1OyLXpa2L_pi9fM8IQQ",
  "custom:issuer_idp": "https://webauth-test.azero.veneto.it/idp/metadata",
  "sub": "alb0a30b-d96a-4166-abc2-bfb9d170a2e8",
  "cognito:groups": [
    "eu-central-1_HFKeJNvjn_AZeroShibProxy"
  ],
  "email_verified": false,
  "custom:codice_struttura": "050511",
  "custom:codice_fiscale": "PPPNNN00M00A111X",
  "iss": "https://cognito-idp.eu-central-1.amazonaws.com/eu-central-1_HFK",
  "cognito:username": "azeroshibproxy_ve3dq6apfwmxae5oikp5bc72rs233keq",
  "given_name": "Test nome",
  "custom:user_id_idp": "Test user id idp",
  "aud": "17uf34ptjs4229a974b0pamqog",
  "identities": [
    {
      "userId": "VE3DQ6APFWMXAE5OIKP5BC72RS233KEQ",
      "providerName": "AZeroShibProxy",
      "providerType": "SAML",
      "issuer": "https://apms-svi.azero.veneto.it/idp/shibboleth",
      "primary": "true",
      "dateCreated": "1600771241466"
    }
  ],
  "token_use": "id",
  "auth_time": 1601540467,
  "apms:gruppi": "op_SIRMI,am_SIRMI",
  "apms:ufficio": "0010",
  "apms:provincia": "VE",
  "apms:ruolo_org": "R.1.1",
  "exp": 1601544068,
  "custom:endpoint_idp": "https://trustapp-test.azero.veneto.it/ws",
  "iat": 1601540468,
  "family_name": "Test cognome"
}
```

4.1.4 Ulteriori approfondimenti su APMS

Il flusso di autenticazione segue l'authorization flow di OIDC (Openid Connect) con l'uso del PKCE (Proof Key for Code Exchange). Nel caso sia richiesto, il meccanismo di autenticazione deve prevedere l'utilizzo del refresh token per evitare che ogni ora, durata dell'ID_Token, l'utente sia costretto ad autenticarsi di nuovo. Sia l'ID_Token sia il refresh token devono essere gestiti in modo da evitare vulnerabilità della sicurezza per l'applicazione a fronte di attacchi come, ad esempio, Cross Site Scripting (XSS) e Cross Site Request Forgery (CSRF).

In caso di front end di tipo SPA è possibile utilizzare anche delle componenti di back end che partecipino al processo di autenticazione (ad es. il BFF) in modo da abilitare l'utilizzo dei cookie per la conservazione delle informazioni critiche per la sicurezza.

4.2 SAML Token

Quando necessario l'applicazione deve essere in grado di ricavare il SAML Token dal sistema IAP tramite transazione RVE-1.b (cfr. documentazione di riferimento). Le informazioni necessarie al recupero del SAML Token sono contenute nell'ID-Token ottenuto durante il processo di autenticazione.

4.2.1 Accesso ai servizi infrastrutturali

L'Utente Federato IAP ha la possibilità di accedere ai servizi infrastrutturali quali Anagrafe Assistiti, Fascicolo Sanitario Elettronico Regionale (FSER), ecc. L'accesso a questa tipologia di servizi è possibile se si è in possesso del SAML Token (si veda il documento Infrastruttura di sicurezza GDL-O di riferimento del sistema di autenticazione IAP). Per ottenere il SAML Token necessario all'accesso, l'applicazione deve contattare l'Identity Provider sul quale l'utente si è autenticato ed invocare la transazione RVE-1.b (si veda la documentazione di riferimento). L'endpoint dell'Identity Provider da utilizzare per recuperare il SAML Token è inserito nel Token Autorizzativo durante la fase di autenticazione (questo contatto è indicato nel diagramma di Figura 5 con la connessione tra APP, l'applicazione, e IAP).

Per approfondimenti in relazione alla "Attestazione di conformità alle specifiche tecniche" vedere § [5.2](#)

4.3 Google Cloud Identity

Azienda Zero, Regione del Veneto e le Aziende Sanitarie del Veneto utilizzano Google Workspaces come strumento di collaborazione. È quindi possibile come opzione di autenticazione per i dipendenti delle sopracitate entità l'uso della Google Cloud Identity.

Nella scelta di questa soluzione va posta molta attenzione ai seguenti fattori:

1. Questa soluzione risulta di più facile e veloce implementazione rispetto ad APMS
2. Rispetto ad APMS non permette di autenticare quegli utenti e operatori sanitari del territorio che non hanno un account Google Workspaces.
3. Rispetto ad APMS questa soluzione crea un "vendor locking"
4. Rispetto ad APMS non permette il single sign on con altre applicazioni

Visti i punti sopra descritti Azienda Zero richiede che venga motivato l'uso di questa soluzione e che l'applicazione permetta altre soluzioni di "social login" oltre a google.

5 VINCOLI PROGETTUALI

In questo capitolo sono elencati dei requisiti che i progettisti devono seguire quando sviluppano i componenti dell'architettura a microservizi descritta in questo documento. In alcuni casi, oltre ai requisiti stessi, sono forniti esempi di com'è possibile implementarli per rispettare i requisiti specificati.

Oltre ai vincoli progettuali esposti in questo capitolo bisogna considerare anche quelli, di carattere più generale, esposti al §. 2 in cui si è discusso dei concetti di base di un'architettura a microservizi.

5.1 Strategia Cloud Italia, Linee Guida AgID e ACN

Centrale per Azienda Zero nel perseguimento dei propri obiettivi è la conformità alla Strategia Cloud Italia e alle linee guida rilasciate dagli enti AgID e ACN. I principali documenti sono disponibili, nella loro versione più aggiornata, ai seguenti indirizzi (non si esclude che possano essere integrati da ulteriori documenti, o che ne richiamino altri):

- <https://innovazione.gov.it/dipartimento/focus/strategia-cloud-italia/>
- <https://www.agid.gov.it/it/linee-guida>
- <https://www.acn.gov.it/portale/crittografia>

5.2 Attestazione di conformità alle specifiche tecniche (ex processo di labeling)

La procedura di attestazione di conformità alle specifiche tecniche consente di verificare il corretto sviluppo degli applicativi che necessitano di accesso ai sistemi centralizzati di Azienda Zero (ad esempio, FSSE0, Anagrafe0, SAR).

La verifica di conformità è gestita dal Consorzio Arsenà, che fornisce la documentazione necessaria e supporta il fornitore durante la fase operativa.

Questa procedura permette inoltre di ottenere i codici applicativi e i certificati indispensabili per la comunicazione con i sistemi di accreditamento forniti da Arsenà.

Dopo aver ricevuto dal Consorzio Arsenà.IT un report con esito positivo e l'identificativo univoco del software, denominato "ApplicationID", Azienda Zero provvede all'emissione del certificato applicativo di produzione. Il certificato, l'ApplicationID e il relativo contesto sono necessari per accedere ai servizi centralizzati di Azienda Zero nell'ambiente di produzione.

Le specifiche di integrazione sono contenute nel documento *Infrastruttura di sicurezza GDL-O Sicurezza*.

5.3 Ambienti di Rilascio

In Azienda Zero sono disponibili vari ambienti per il rilascio applicativo, di seguito sono indicati i principali e il loro scopo.

Sigla	Nome	Descrizione	Dato
PROD (o nessuna)	Produzione	Ambiente unico per l'erogazione del servizio all'utente finale	Dati reali di produzione
LAB	Labeling/Pre-produzione	Ambiente simile alla produzione per: • versione • infrastruttura • dato, ma anonimizzato	Dati reali ma anonimizzati Dati generati
TEST	Test	Ambiente di test per la messa in produzione di bug-fix e nuove funzionalità per il collaudo. Tale ambiente è anche a disposizione di applicativi terzi che sono in fase di sviluppo delle integrazioni (vedi sviluppo). Questo ambiente deve essere utilizzato per altri scopi come: • Formazione (FOR) • Collaudo (COL) • Test di Carico • altro	Dati generati o dati reali ma anonimizzati
SVI	Sviluppo	Ambiente per lo sviluppo delle integrazioni verso sistemi/servizi di Azienda Zero non raggiungibili dall'ambiente di sviluppo del fornitore.	Dati generati

Gli ambienti di Produzione e almeno uno tra Sviluppo e Test sono obbligatori, sebbene l'utilizzo di tre ambienti sia preferibile.

L'ambiente di Sviluppo è necessario quando si opera direttamente nell'infrastruttura di Azienda Zero o quando l'applicativo richiede integrazioni con i suoi sistemi non raggiungibili tramite l'ambiente di sviluppo del fornitore.

Un ambiente di Labeling è richiesto se l'applicazione è necessaria per il labeling di nuovi servizi di terzi, previa autorizzazione di Azienda Zero tramite il Portale Rilasci.

Il rilascio di applicazioni nei diversi ambienti segue un flusso approvativo generale così sintetizzato:

- Ambienti di Sviluppo: il rilascio è gestito dal fornitore di sviluppo, o tramite automazione collegata al commit, o previa attivazione delle pipeline di build e deploy.
- Ambienti di Test e Produzione: il rilascio è avviato manualmente da IT Operations dopo che la richiesta, formalizzata tramite il Portale Rilasci, è stata autorizzata da Azienda Zero.

5.4 Durata Sessione Utente

La durata della sessione utente è determinata dal periodo di validità del Token Autorizzativo (ID_Token). È possibile estendere la sessione utilizzando un Refresh Token, la cui validità è configurabile, di solito in giorni o ore.

Oltre a questi vincoli di durata, la sessione utente viene invalidata nei seguenti casi:

- Disconnessione dell'utente.
- Inattività dell'utente per più di [Session Timeout], valore indicato 30 minuti.
- Connessione dell'utente oltre il [Session Expiration Time], valore indicato 12 ore.

[Session Timeout] è nell'ordine delle decine di minuti, mentre [Session Expiration Time] è nell'ordine delle ore. Entrambi i valori devono essere concordati con il referente del progetto. Alla scadenza della sessione, l'utente deve autenticarsi nuovamente per continuare a utilizzare l'applicazione.

5.5 Role Based Access Control (RBAC)

Per abilitare i microservizi al RBAC (Role Based Access Control) e, più in generale, per fornire ai microservizi le informazioni utili ad una corretta gestione del flusso elaborativo, è necessario che le informazioni contenute nel Token Autorizzativo ottenuto in fase di autenticazione ed altri dati ausiliari che si vedranno tra poco, siano in qualche modo propagati attraverso tutti i componenti dell'architettura.

Le informazioni contenute nel Token Autorizzativo (cfr. § [4.1.3](#)) da fornire ai microservizi sono:

- codice fiscale utente
- nome utente
- cognome utente
- codice struttura
- gruppi ed attributi forniti dal sistema APMS
- identificativo dell'IdP di autenticazione
- endpoint RVE-1.b dell'IdP di autenticazione
- userid sull'IdP di autenticazione

Inoltre, per permettere ai microservizi di produrre un log applicativo più semplice da analizzare è necessario propagare anche i parametri:

- identificativo applicazione - è un codice univoco che identifica l'applicazione

- identificativo sessione - è un codice univoco che identifica la sessione dell'utente, può essere definito a valle del processo di autenticazione
- identificativo operazione - è un codice univoco dell'operazione effettuata dall'utente, in un'applicazione web potrebbe essere un codice associato a ciascuna azione (click) che l'utente esegue sul browser

Gli identificativi di sessione e di operazione hanno lo scopo di correlare le righe di log prodotte dai microservizi attivati dalle operazioni dell'utente. Si ricorda che in un'architettura a microservizi i log sono prodotti da componenti separate ed è quindi importante avere dei parametri che permettano di correlare log diversi.

Per motivi legati alla sicurezza le informazioni elencate nel primo gruppo, quelle contenute nell'ID-Token, non possono essere passate esplicitamente al back end. Il Token Autorizzativo deve raggiungere integro il back end dove deve essere validato (ad es. con la Lambda Authorizer montata sull'API Gateway nell'architettura in cui il front end è una SPA). Dopo la validazione, quando il flusso è ormai in esecuzione sul back end, le informazioni possono anche essere estratte dal Token Autorizzativo per essere passate ai microservizi in un formato più efficiente da gestire (ad es. tramite opportuni header HTTP). Le informazioni del secondo gruppo possono essere passate in modo esplicito a partire dal front end (ad es. tramite opportuni header HTTP). Tutte le informazioni elencate sopra, quelle di entrambi i gruppi, devono essere propagate anche nelle chiamate tra microservizi, siano queste di tipo sincrono o di tipo asincrono (cfr. § [2.4](#)).

5.6 Gestione stringhe di testo

Le stringhe di testo generate dal sistema devono essere rappresentate attraverso codici, in modo che l'associazione tra il codice generato dal sistema e il testo effettivo da visualizzare nell'Interfaccia Utente possa essere gestita in modo separato dal codice sorgente.

Per entrare nel dettaglio è utile suddividere le stringhe di testo nelle categorie seguenti:

- **statiche:** sono informazioni che non sono associabili ai dati trattati dal sistema; generalmente sono scolpite direttamente nel codice dell'Interfaccia Utente (testi statici, label, tooltips, etc.)
- **testi di input:** sono testi associati a dati da utilizzare nella fase di acquisizione dati nel caso si voglia far scegliere all'utente uno dei possibili valori di input e non un testo libero (valori di elementi di liste o di bottoni mutuamente esclusivi, etc.). Sono stringhe di testo che traducono il valore utilizzato per conservare l'informazione nel sistema di persistenza nel linguaggio dell'utente. Ad esempio: il sistema accetta per il campo "Priorità" i valori 1, 2 e 3 che all'utente devono essere presentati come "Bassa", "Media" ed "Alta".
- **testi di output:** sono testi associati a dati restituiti dalle componenti di back end. Sono stringhe di testo che traducono il valore utilizzato per conservare l'informazione nel sistema di persistenza nel linguaggio dell'utente. Ad esempio: la componente di back end restituisce per l'attributo "Priorità" i valori 1, 2 e 3 che all'utente devono essere presentati come "Bassa", "Media" ed "Alta".
- **messaggi:** sono testi ottenuti da una concatenazione di più parti alcune delle quali possono essere fisse ed altre, invece, possono essere determinate a partire da codici restituiti dal back end, ad esempio: "La domanda N° Prot. PR001 non è stata accolta

perché la categoria EX DIPENDENTE associata al richiedente non è ammessa”. In questo esempio si possono individuare tre parti distinte che concorrono alla formazione del testo:

- “La domanda N° Prot. \$1 non è stata accolta perché la categoria \$2 associata al richiedente non è ammessa”: il testo base del messaggio, ovvero, la sua parte fissa in cui sono stati usati dei segnaposto (placeholder) per le parti variabili.
- PR001: un valore che è frutto di un’elaborazione restituito direttamente dal back end.
- EX DIPENDENTE: la descrizione di un valore che viene restituito dal back end in forma di codice (es. EX DIPENDENTE = 2).

Nei paragrafi che seguono è descritta nel dettaglio la gestione delle diverse categorie di stringhe di testo appena individuate.

5.6.1 Statiche

La gestione è totalmente a carico del front end secondo quelle che sono le best practice della tecnologia utilizzata.

5.6.2 Testi di input

Per quanto detto sopra, questo è il caso in cui l’utente deve inserire a sistema un dato scelto tra una lista di possibili valori. Nel caso in cui i valori da mandare alle componenti di back end non corrispondano con le descrizioni visualizzate, il front end è dotato di una tabella di decodifica dalla quale recupera le stringhe di testo da visualizzare. Le chiavi utilizzate nella tabella di decodifica sono i valori gestiti dal back end.

5.6.3 Testi di output

I testi di output sono quelli associati a valori restituiti dal back end che non corrispondono alle stringhe da visualizzare. Ad esempio, il back end potrebbe restituire lo stato di lavorazione associato ad un’istanza con il valore numerico 1 che all’utente deve essere presentato con la descrizione “presa in carico”. Come già detto per i testi di input, per questi casi il front end è dotato di una tabella di decodifica dalla quale recupera le stringhe di testo da visualizzare utilizzando come chiavi i valori provenienti dal back end.

5.6.4 Messaggi

Come detto sopra, il messaggio nasce dalla concatenazione di più parti di testo. È il back end che restituisce le indicazioni al front end su come visualizzare il testo che andrà a comporre l’intero messaggio.

Un esempio di struttura dati che il back end potrebbe produrre in questo caso è il seguente:

```
{
  "codice": "[codice messaggio]",
  "parametri": [
    {
      "tipo": "[tipo parametro]",
      "valore": "[valore parametro]",
      "timestamp": "[timestamp decodifica parametro]"
    }
  ]
}
```

[codice messaggio] - il codice univoco associato alla parte fissa del messaggio la cui decodifica è effettuata dal front end tramite tabella di decodifica. Il testo del messaggio non viaggia nella struttura, viaggia solo il suo codice ed è responsabilità del front end convertire il codice nel testo corrispondente.

[parametri] - in un messaggio, individuato univocamente da [codice messaggio] e nel quale sono previste delle parti variabili, generalmente indicate nel testo del messaggio da segnaposto (placeholder), i parametri sono i valori che queste parti variabili devono assumere. Ad esempio, se il testo del messaggio con [codice messaggio] = MSG0010 è "La domanda N° Prot. \$1 non è stata accolta perché la categoria \$2 associata al richiedente non è ammessa"; allora, nella struttura dell'esito, per questo messaggio ci sono due parametri: il numero protocollo e la categoria; nell'ordine indicato dall'indice dei segnaposto \$x. I parametri sono un array di items aventi la struttura seguente.

[tipo] - il tipo del parametro. Nell'esempio in cui il messaggio è "La domanda N° Prot. \$1 non è stata accolta perché la categoria \$2 associata al richiedente non è ammessa"; ci sono due tipi di parametri, il protocollo, che è una sequenza di caratteri (una stringa) e la categoria, entrambi determinati nel corso del flusso elaborativo, magari presi dal database. Mentre il protocollo così com'è stato letto dal database può essere sostituito al segnaposto \$1, la categoria potrebbe persistere nel database in forma di codice (es. "EX DIPENDENTE" = 2) e quindi è necessario recuperarne la descrizione prima di poter effettuare la sostituzione del segnaposto \$2. Il tipo del parametro, quindi, sta ad indicare al front end come ottenere il testo da sostituire al segnaposto (quale tabella di decodifica utilizzare). Per i parametri che di per sé già rappresentano un valore, come il protocollo nell'esempio fatto, il tipo del parametro è sempre: valore. Per tutti quelli che necessitano di una decodifica il tipo del parametro è associato al tipo di dato a cui fa riferimento il parametro. Nell'esempio fatto il tipo del parametro per la categoria potrebbe essere: catRich (categoria richiedente); e, noto il codice del parametro (vedere [valore]), tanto basta al front end per accedere alla corretta tabella di decodifica per recuperare la descrizione della categoria.

[valore] - il valore del parametro. Per i parametri con [tipo] = valore è esattamente la sequenza di caratteri da sostituire al segnaposto. Per [tipo] <> valore è il codice univoco del parametro nella sua tabella di decodifica. Nell'esempio fatto sopra il parametro della categoria ha valore 2.

[timestamp] - timestamp per la decodifica del testo del parametro. Non è obbligatorio. In alcuni casi il codice del parametro da solo non è sufficiente a recuperare la corretta descrizione, ad esempio, per ottenere la descrizione del codice catastale del comune italiano E472 è necessario sapere se si fa riferimento al periodo in cui il comune si chiamava Littoria o a quello in cui si chiama Latina.

5.7 Gestione degli errori

Per errore si intende un qualsiasi evento anomalo che interrompe il flusso elaborativo prima che questo venga completato. Esempi di errore sono:

- La temporanea indisponibilità di una o più parti dell'infrastruttura
- La presenza di un bug: ad esempio il tentativo di scrivere un valore nel database che non è compatibile con il campo che dovrebbe ospitare; un null pointer; etc...
- L'avvio di un'operazione su un microservizio con dati di input errati: dati obbligatori mancanti; dati di input incompatibili (es. data da maggiore di data a)

Quelli appena elencati, insieme a qualsiasi altra causa che provoca l'interruzione imprevista del flusso elaborativo, sono considerati, in questo ambito, errori.

Di contro, non è considerato un errore l'esito negativo di un'elaborazione (cfr. § [5.8](#)).

Le regole da seguire nella gestione degli errori sono le seguenti:

1. gli errori non sono gestiti (try catch) a meno che non sia necessario (ad esempio per soddisfare un requisito specifico).
2. nei componenti che costituiscono il back end (microservizi) il verificarsi di un errore è sempre registrato nel log applicativo con il livello di errore appropriato (cfr. § [3.7.3](#)).
3. i componenti di back end restituiscono codice di ritorno HTTP e descrizione coerenti con l'errore che si è verificato ad eccezione dei codici 5XX. In quest'ultimo caso la descrizione dell'errore generata in automatico dal sistema deve essere mascherata da una descrizione generica del tipo "Errore interno". Deve comunque essere inoltrato lo status code al client.
4. Il messaggio mostrato dal front end all'utente va concordato con il referente dell'applicazione. In alcuni casi la scelta potrebbe essere quella di non fare differenza tra i tipi di errore e mostrare all'utente un'unica tipologia di messaggio: "Sistema momentaneamente non disponibile". Al messaggio è possibile aggiungere l'identificativo operazione (cfr. § [5.5](#)) che, eventualmente, l'utente può comunicare all'help desk per facilitare la risoluzione del problema. In ogni caso il messaggio non deve rilasciare informazioni utili a identificare l'infrastruttura tecnologica che eroga il servizio come ad esempio versioni, nomi di prodotti, ip, nomi dns.
5. Le pagine di errore destinate all'utente finale o in generale visibili dall'esterno devono essere prive di informazioni che possano ricondurre alle tecnologie e infrastrutture che supportano il servizio. In tal senso le pagine web (ma anche le risposte API) devono essere standardizzate in modo da offrire solo i contenuti necessari per gestire l'errore. Nelle applicazioni web server side l'attività di occultare i messaggi di errore applicativi potrebbe essere eseguita dai sistemi di bilanciamento e protezione.

Di seguito un esempio delle informazioni richieste ad una pagina di errore:

- a. Codice html dell'errore
- b. Descrizione sintetica dell'errore
- c. Identificativo Operazione
- d. Identificativo Applicazione

5.8 Esito di un'operazione

Nell'architettura presentata in questo documento come linea guida con il termine esito si intende indicare il risultato dell'elaborazione eseguita da un'operazione di un microservizio qualora non si verificano errori (cfr. § [5.7](#)). In generale l'esito è legato ad operazioni di back end di tipo SOA (cfr. § [2.6](#)).

Come esempio di esito negativo si può pensare ad un ipotetico sistema di prenotazione di una terapia in cui si prova a selezionare una data che non è più disponibile; in tal caso l'utente viene avisato con un opportuno messaggio che l'operazione non può essere completata. Per quanto detto all'inizio di questo paragrafo non si sono verificati errori e l'impossibilità di completare l'operazione è frutto dell'esecuzione completa del flusso elaborativo nel quale è gestito il caso di disponibilità esaurita.

Per maggiore chiarezza si riporta l'esempio di una possibile struttura dati che il back end, in aggiunta all'output specifico dell'operazione, potrebbe restituire al front end per comunicare l'esito di un'operazione.

```
{
  "esito": {
    "codice": "[codice ritorno]",
    "operationId": "[operation id]",
    "messaggi": [
      {
        "severita": "[severità]",
        "codice": "[codice messaggio]",
        "parametri": [
          {
            "tipo": "[tipo parametro]",
            "valore": "[valore parametro]"
          },
          {
            "timestamp": "[timestamp decodifica parametro]"
          }
        ]
      }
    ]
  }
}
```

[codice ritorno] - può assumere i valori:

- E1: Operazione eseguita – nessun messaggio o messaggi di [severità] = S1
- E2: Operazione eseguita con avvisi - al più ci sono messaggi di [severità] = S2
- E3: Operazione non eseguita – c'è almeno un messaggio di [severità] = S3

con ovvio significato rispetto all'esito dell'operazione.

[operation id] - identificativo univoco dell'operazione. Ogni chiamata ai servizi di back end ha un codice univoco dell'operazione (cfr. § [5.5](#)).

[messaggi] - array di items aventi la stessa struttura indicata per i messaggi al § [3.6](#) con in più l'attributo severità descritto di seguito.

[severità] - può assumere i seguenti valori:

- S1: Informazione
- S2: Avviso
- S3: Impedimento

5.9 API

La progettazione delle API, in primo luogo, deve attenersi al paragrafo § [5.1](#)

I microservizi realizzati devono essere accompagnati dalla documentazione secondo le specifiche correnti OpenAPI.

Lo swagger file verrà richiesto in fase di primo deploy applicativo e verrà da prima utilizzato per la configurazione degli API Gateway e quindi, se necessario, pubblicato sul catalogo delle API.

5.10 Healthcheck

I microservizi realizzati devono esporre metodi che consentano la verifica automatica dello stato di funzionamento del microservizio. Per il formato di questi metodi e strutture si faccia riferimento alla documentazione di [Spring Boot Actuator](#). Si ponga particolare attenzione a inibire endpoint che possano compromettere il corretto funzionamento del componente (ad esempio /shutdown) o che esponano informazioni relative all'infrastruttura tecnologica.

Tale richiesta vale anche per le eventuali applicazioni monolitiche, in questo caso l'healthcheck dovrà rappresentare lo stato di salute dell'applicativo e delle sue principali dipendenze.

5.11 Interfaccia Utente

L'Interfaccia Utente (UI - User Interface) deve essere aderente alle specifiche AgID (ad esempio, ma non limitatamente, a quelle relative all'accessibilità) vedere paragrafo § [5.1](#).

5.12 Feature Toggle e API Version

Durante il ciclo di vita di una applicazione, soprattutto nello sviluppo di microservizi con logica di gestione dei repository di sorgenti *master-first*, è molto probabile scontrarsi con l'esigenza di avere molteplici sviluppi di correttive e nuove funzionalità in contemporanea. Per gestire tale esigenza si suggerisce di usare la metodica [Feature toggle](#) o detta anche Feature Flag.

Di seguito un esempio per springboot <https://www.baeldung.com/spring-feature-flags>

In sintesi si prevede l'uso di flag a codice che attivano o disattivano le nuove funzionalità o correzioni. Tali flag possono essere presentati all'utilizzatore tramite un'opzione da attivare e che gli permetterebbe di accedere ad una versione dell'applicazione in cui sono attive una o più funzionalità da testare, nel backend tale opzione che permette di attivare e disattivare alcune funzionalità andrebbe gestita tramite variabili o properties passate all'applicativo dall'ambiente di esecuzione (es. configmap o secret di Kubernetes) o tramite header impostati dal reverse proxy o dalla componente di front end.

L'uso di tale metodologia permette, inoltre, di creare in un singolo ambiente più tenant paralleli come test, training, pre produzione, quality, etc. semplicemente gestendo la possibilità di attivare "flag" da parte di utenti che devono accedere a parti di applicazione specifiche.

A compendio delle metodologia sopra descritta vi è l'uso delle versionamento delle API per mettere a disposizione nuove funzionalità non retrocompatibili, l'API Gateway gestisce nativamente la funzione e a livello di sviluppo è necessario implementare diversi backend per rispondere alle diverse versioni oppure lo stesso backend deve gestire le due versioni in contemporanea con il metodo del feature toggle.

È importante prevedere un ciclo di vita delle versioni software che permetta di dismettere le funzioni o le versioni obsolete in un tempo contenuto.

Alcuni esempi:

Variabile generica di ambiente:

nome: AMBIENTE

value: [svi || test || prod]

nome: CONFIG_APP_FUNZIONEX

value: [TRUE || FALSE]

5.13 Risorse esterne e servizi terzi

Nell'ambito della normativa vigente relativa alla protezione dei dati personali, non è consentito l'uso di risorse e servizi digitali erogati da paesi terzi che determinino flussi di dati al di fuori dell'Unione Europea e dello Spazio economico europeo (SEE).

Qualora l'uso di servizi con tali caratteristiche fosse inderogabile, è indispensabile darne preventiva comunicazione ad Azienda Zero per consentire l'opportuna valutazione di eventuali misure correttive che riconducano il trattamento dei dati personali al rispetto della normativa vigente.

5.14 Protocolli di comunicazione

Tutte le comunicazioni di rete, come linea guida generale, ove possibile devono utilizzare adeguati protocolli di cifratura, sono comprese sia le comunicazioni esposte che quelle interne, a solo scopo esemplificativo possiamo identificare le comunicazioni verso le base dati, tra servizi applicativi e tra sistemi di bilanciamento e le applicazioni.

La sicurezza delle comunicazioni diventa obbligatoria nelle comunicazioni tra datacenter diversi e tra enti o strutture diverse. A titolo di esempio devono essere securizzate le comunicazioni tra i data center in Cloud, PSN, PSR, data center di Azienda Zero e Aziende Sanitarie del territorio.

Tutte le comunicazioni che sfruttano il canale internet devono essere securizzate indipendentemente dal dato trasmesso.

Tutte le chiamate web (HTTP) devono usare il protocollo TLS non deprecato e con mTLS su richiesta di Azienda Zero.

Eventuali protocolli di comunicazione proprietari come ad esempio tra apparati medicali e i sistemi centralizzati o connessioni Jdbc dovranno essere autorizzati da Azienda Zero e, in ogni caso, cifrati.

Inoltre nei casi di comunicazioni in rete wan o internet è opportuno valutare se introdurre la compressione per ottimizzare il carico sulle reti.

In riferimento al protocollo HTTPS è essenziale che l'applicazione gestisca il binding su più url diverse, questo è dovuto all'infrastruttura di rete privata tra le aziende sanitarie nella quale, a titolo esemplificativo e non esaustivo, le applicazioni vengono esposte con l'url `https://applicazione.extra.azero.veneto.it` mentre in internet lo stesso applicativo verrebbe esposto con `https://applicazione.azero.veneto.it`.

5.15 Certification Authority e Certificati mTLS e TLS

Allo scopo di proteggere il dato in transito tutti i servizi devono essere esposti tramite protocolli sicuri ed in particolare i servizi web devono essere esposti tramite il protocollo HTTPS. Alcuni servizi, in aggiunta, devono essere esposti tramite la soluzione di autenticazione dei certificati mTLS (mutua autenticazione). Per i servizi ospitati dai sistemi di Azienda Zero la gestione delle esposizioni sono gestite per mezzo dei sistemi di sicurezza e bilanciamento e quindi non sono in carico al fornitore software, diverso il discorso per i servizi SaaS o assimilabili.

5.15.1 Certification Authority

Azienda Zero per i servizi esposti utilizza CA pubbliche e riconosciute (trusted) dai browser e software che fanno uso di trust store o key store. La gestione dei certificati server rilasciati da tali CA è in carico all'IT Operations di Azienda Zero per tutti i servizi interni, mentre deve essere coordinata per i servizi che i fornitori vogliano offrire in modalità SaaS.

Azienda Zero inoltre utilizza una CA privata per la generazione dei certificati client per l'autenticazione di alcuni servizi interni come ad esempio FSER. Trattandosi di una CA non riconosciuta pubblicamente è necessario che le applicazioni che debbano farne uso ne importino la parte pubblica nei propri trust store o key store. Nella maggior parte dei casi sarà l'IT Operations che provvedere a mettere a disposizione il file nei file system delle Virtual Machine. Diverso il caso delle applicazioni containerizzate, eventuali certificati di cui effettuare il trust devono essere inseriti in fase di build a livello di sistema operativo (utilizzando update-ca-certificates), in modo da predisporre un'immagine che includa già tutti i certificati di cui necessita.

5.15.2 mTLS

L'autenticazione tramite certificato client, come descritto sopra, è necessaria per l'accesso a servizi di vari circuiti come FSER. Il fornitore di applicativi software che devono accedere a tali circuiti deve prevedere questo livello di autenticazione che in alcuni casi fa anche da soluzione di autorizzazione e profilazione. I certificati verranno rilasciati a seguito di compilazione della documentazione adeguata e di eventuale labeling (cfr. § [5.2](#))

5.16 Docker Base Image

I docker devono essere creati sulla base della "Red Hat Universal Base Image" scaricabile dal sito di [Red Hat](#) o "amazoncorretto" scaricabile dal repository Dockerhub [amazoncorretto](#)

La standardizzazione delle immagini di base permette di aumentare la stabilità, affidabilità e sicurezza delle applicazioni e di ridurre gli sforzi di manutenzione.

Scostamenti dalle indicazioni di cui sopra verranno valutate su richiesta del fornitore.

5.17 Affidabilità, Resilienza e Continuità Operativa

L'affidabilità, la resilienza dei servizi e la continuità operativa rappresentano vincoli imprescindibili che il fornitore applicativo è tenuto a considerare durante la fase di progettazione e sviluppo di architetture software nuove o nelle manutenzioni evolutive di quelle esistenti.

5.18 Ambienti non di produzione

Le applicazioni negli ambienti non di produzione devono rispettare i seguenti vincoli di sicurezza:

1. **Esposizione e indicizzazione:** Non devono essere esposte su Internet, se non in casi di stretta necessità. In questi casi, devono essere escluse dall'indicizzazione da parte di sistemi esterni (es. motori di ricerca, motori AI, ecc.).
2. **Autenticazione:** Devono implementare un sistema di autenticazione, come specificato nel capitolo § [4](#).
3. **Dati:** Non devono contenere dati di produzione e non devono accedere a dati di

produzione.

4. **Servizi:** Non devono erogare servizi di produzione.
5. **Identificazione dell'ambiente:** Devono utilizzare un URL che indichi chiaramente l'ambiente (cfr. § [5.3](#)) ed essere caratterizzate graficamente per distinguere l'ambiente di produzione da quello non di produzione.

5.19 Limiti invio E-Mail

L'invio di email da parte delle applicazioni è un processo centralizzato e gestito da IT Operation, distinguendo due categorie principali:

1. **Notifiche a liste fisse:** Si tratta dell'invio di notifiche a un numero predefinito di indirizzi email, come gli utenti autenticati delle console di amministrazione del servizio. Per questa tipologia, è sufficiente l'uso di protocolli di comunicazione sicuri (es. HTTPS per le API AWS SES o SMTPS per i protocolli tradizionali).
 La gestione degli indirizzi email interni all'applicazione deve essere soggetta a revisione periodica e seguire un flusso di abilitazione e disabilitazione che ne garantisca il ciclo di vita.
2. **Invio a liste dinamiche (mailing list):** Riguarda l'invio di email a liste dinamiche generate, ad esempio, dalla registrazione di utenti al servizio. In questo caso, oltre alla sicurezza dei protocolli, è fondamentale prevedere in fase di progettazione un sistema per la verifica delle email inserite dagli utenti e il monitoraggio continuo delle email di "bounce" e degli errori di invio. Questo è cruciale per prevenire il superamento di soglie di errore che potrebbero comportare l'inserimento in blacklist.

5.20 Limiti di accesso a base dati di altri servizi

Come descritto nel paragrafo [2.3](#), il dominio del dato deve essere rispettato. Ogni componente applicativa (microservizio) deve quindi accedere in modo esclusivo ai propri dati, che a livello di progettazione architetturale del software devono essere separati dagli altri domini di dato.

Per necessità specifiche in cui un servizio debba accedere a più domini di dati (ad esempio, per reportistica o estrazioni periodiche), sono disponibili diverse soluzioni:

1. **Accesso tramite microservizio:** I dati vengono comunque acceduti tramite chiamate al microservizio, e l'interrogazione e aggregazione dei dati sono gestite a livello applicativo.
2. **Infrastrutture di virtualizzazione del dato:** Si possono utilizzare infrastrutture di virtualizzazione del dato e sistemi di reportistica che non sono soggetti ai vincoli applicativi.
3. **Uso di ETL o soluzioni analoghe:** È possibile ricorrere a ETL (Extract, Transform, Load) o

soluzioni simili per elaborare i dati in un contesto separato dall'applicazione principale.

5.21 Exit Strategy dai sistemi SaaS

Le soluzioni SaaS proposte devono prevedere una chiara strategia di uscita (Exit Strategy) che assicuri ad Azienda Zero la piena titolarità delle funzionalità e dei dati contenuti nel servizio. Questa strategia dovrà essere presentata insieme alla proposta tecnica contrattuale.

6 CI/CD

Al fine di velocizzare il rilascio di nuove versioni software, il team ITOperation si occupa dell'implementazione di pipeline automatizzate che gestiscono la compilazione, la verifica e il deployment di ciascun progetto. È quindi indispensabile che i fornitori di applicazioni adottino strumenti di compilazione standardizzati e definiscano con chiarezza il processo di distribuzione degli artefatti, garantendo così l'efficacia delle pipeline.

6.1 Continuous Integration

La sezione "Continuous Integration" comprende tutte le attività di compilazione e test automatici. Queste attività sono essenziali per garantire la consistenza del pacchetto applicativo generato e il corretto funzionamento del servizio durante il deployment. A tal fine, è fondamentale fornire indicazioni chiare riguardo la compilazione e i test automatici da eseguire in questa fase.

Gli strumenti per la compilazione e l'esecuzione dei test automatici devono rispettare i seguenti requisiti:

- **Disponibilità e Costi:** Devono essere liberamente disponibili, senza la necessità di sottoscrizioni o oneri per Azienda Zero. Si privilegiano strumenti open source standard di mercato. Qualora non fosse possibile utilizzare uno strumento open source o liberamente disponibile, lo strumento dovrà essere fornito direttamente ad Azienda Zero, che ne deterrà il diritto d'uso senza oneri aggiuntivi. Alcuni esempi di strumenti attualmente in uso includono:
 - Java: Maven, Gradle
 - Node.js: npm
 - Python: pytest
- **Compatibilità Ambiente:** Gli strumenti dovrebbero essere eseguiti in un ambiente Linux. È preferibile che siano pacchettizzati in container per facilitare l'integrazione e l'utilizzo all'interno degli strumenti di esecuzione delle pipeline.
- **Esecuzione Non Interattiva:** Devono poter essere eseguiti da riga di comando in modalità non interattiva. Il comando di compilazione, specificato nel file `README.md` nel repository Git, deve essere eseguibile senza supervisione o inserimento manuale di parametri. Tutti i parametri necessari devono essere passati tramite argomenti della command line o recuperati dall'applicazione direttamente dall'ambiente.
- **Gestione Errori:** In caso di fallimento della compilazione o dei test, lo strumento deve restituire codici di errore diversi da zero, riportando su Standard Output/Standard Error la motivazione dell'errore.

In aggiunta, i test possono produrre un report di esecuzione in formato *junit xml*. Questo formato consente l'analisi automatica dei risultati dei test e la loro visualizzazione diretta nell'interfaccia dello strumento.

6.2 Continuous/Automated Deployment

Dopo la compilazione degli artefatti e l'esecuzione dei test automatici, si procede con il deploy dell'artefatto. Le istruzioni per il deploy devono essere specificate nel file README.md. Sebbene le modalità di deploy varino in base all'ambiente target (es. Tomcat e Kubernetes), devono rispettare requisiti generali:

- **Non interattivo:** Le operazioni di deploy devono essere completamente eseguibili da riga di comando, senza richiedere alcun input da parte dell'operatore.
- **Rolling deploy:** In caso di più repliche applicative, il deploy deve supportare la modalità rolling, permettendo la coesistenza temporanea di due versioni dell'applicazione. Se ciò non fosse possibile, è necessaria una finestra di fermo applicativo.
- **Interazione con il bilanciatore:** È prevista la possibilità di interagire con l'infrastruttura di bilanciamento per escludere temporaneamente un'istanza dal deployment.

Le modifiche al database dovrebbero idealmente precedere il deploy applicativo, ma sono accettate anche esecuzioni successive. Se non fosse possibile, tali modifiche devono essere automatizzabili ed eseguibili durante la procedura di deploy. A tal fine, si raccomanda l'implementazione di Liquibase, ove applicabile. Qualora le modifiche non fossero compatibili con la nuova versione applicativa, il team IT Operation dovrà essere preavvisato tramite gli strumenti di richiesta dei rilasci applicativi.

7 DOCUMENTAZIONE

Ogni nuovo progetto o manutenzione evolutiva e/o correttiva che impatti sull'infrastruttura deve seguire il flusso di change management in uso presso Azienda Zero. I documenti che il fornitore è tenuto a fornire sono i seguenti:

1. **RFC, Request for Change**

È il documento che operativamente avvia la procedura di Change Management in Azienda Zero. Il documento raccoglie una serie di informazioni utili al fornitore per prendere coscienza dei principali aspetti per il deploy di nuovi applicativi e utile ad Azienda Zero per valutare il perimetro del servizio e valutarne l'impatto sull'infrastruttura.

2. **SA, Scheda Architettuale**

Documento consegnato contestualmente al rilascio del codice e che rappresenta tecnicamente l'infrastruttura applicativa definitiva del prodotto sviluppato. Tale documento verrà poi integrato in collaborazione con IT Operation in caso di modifiche durante la fase di rilascio applicativo.

3. **MI, Manuale d'Installazione**

Documento consegnato prima del rilascio del codice e che dettaglia i prerequisiti tecnici e le parametrizzazione che l'IT Operation dovrà utilizzare per il deploy applicativo.

4. **MU, Manuale Utente**

Documento consegnato prima del rilascio del codice e che dettaglia come l'utilizzatore finale interagirà con il prodotto.

5. **Manuale Operatore e/o Amministratore**

Documento consegnato prima del rilascio del codice e che dettaglia come gli amministratori e gli operatori interagiscono con le interfacce di amministrazione dell'applicativo.

6. **Manuale per il supporto all'utente**

Documento consegnato ad Azienda Zero in concomitanza del deploy applicativo contenente le informazioni necessarie al Service Desk per dare un primo supporto all'utente.

Tale documento deve contenere a titolo di esempio le seguenti informazioni:

- a. Lista codici di errore e relative interpretazioni
- b. Possibili interventi che il service Desk può eseguire per la risoluzione dei problemi
- c. FAQ: lista di possibili domande e relative risposte.

Ogni nuovo progetto, manutenzione evolutiva o correttiva che abbia un impatto sull'infrastruttura deve aderire al flusso di change management di Azienda Zero. Il fornitore è tenuto a fornire i seguenti documenti:

- **RFC (Request for Change):** Questo documento avvia operativamente la procedura di Change Management in Azienda Zero. Contiene informazioni essenziali per il fornitore, utili a comprendere gli aspetti chiave per il deployment di nuove applicazioni, e per Azienda Zero, al fine di valutare il perimetro del servizio e il suo impatto sull'infrastruttura.

- **SA (Scheda Architetture):** Consegnata contestualmente al rilascio del codice, questa scheda rappresenta l'infrastruttura applicativa definitiva del prodotto sviluppato. Verrà integrata in collaborazione con l'IT Operation in caso di modifiche durante la fase di rilascio applicativo.
- **MI (Manuale d'Installazione):** Questo manuale, consegnato prima del rilascio del codice, descrive dettagliatamente i prerequisiti tecnici e le parametrizzazioni che l'IT Operation dovrà utilizzare per il deployment applicativo. Il documento dovrà essere precompilato dal fornitore e verrà poi finalizzato durante il Kickoff con i tecnici di IT Operation.
- **MU (Manuale Utente):** Consegnato prima del rilascio del codice, questo documento descrive come l'utente finale interagirà con il prodotto.
- **Manuale Operatore e/o Amministratore:** Consegnato prima del rilascio del codice, questo manuale dettaglia l'interazione di amministratori e operatori con le interfacce di amministrazione dell'applicativo.
- **Manuale per il supporto all'utente:** Consegnato ad Azienda Zero in concomitanza del deployment applicativo, contiene le informazioni necessarie al Service Desk per fornire un primo supporto all'utente. A titolo esemplificativo, deve includere:
 - Lista dei codici di errore e relative interpretazioni.
 - Possibili interventi che il Service Desk può eseguire per la risoluzione dei problemi.
 - FAQ: lista di possibili domande e risposte.